100 Kafka Interview

Questions & Answers

with Detailed Code Examples

Part 1: Questions 1 – 17

Kafka Core • Architecture • Producers • Consumers • Java API

Spring Boot • Transactions • Error Handling • KRaft

by **TechTribers**

@techtribes • www.techtriber.com

Q1. What is Apache Kafka and how does it differ from traditional message queues like RabbitMQ?

Easy Core

Answer:

Apache Kafka is a **distributed event streaming platform** built around an immutable, append-only commit log. Unlike traditional message queues (store-and-forward, messages deleted after consumption), Kafka **retains messages** for a configurable retention period regardless of consumption status.

Feature	Traditional Queue (RabbitMQ)	Apache Kafka
Data Model	Transient; deleted after ack	Immutable log; retained by time/size
Consumer Model	Competing consumers (one gets msg)	Consumer groups + independent readers
Replay	Not possible after ack	Full replay by resetting offset
Ordering	Per-queue FIFO	Per-partition strict ordering
Throughput	10K-50K msgs/sec	1M+ msgs/sec per cluster
Persistence	Memory-first, optional disk	Disk-first, sequential I/O optimized
Protocol	AMQP, push-based delivery	Custom binary protocol, pull-based

■ **Pro Tip:** Key phrase: *'Kafka is a distributed commit log, not a message queue. It provides durable, ordered, replayable event streams.'* This immediately sets you apart from candidates who say 'it's like RabbitMQ but faster.'

Q2. Explain Kafka's architecture — brokers, topics, partitions, replicas, ISR, and the controller.

Easy Architecture

Answer:

Component	What It Is	Key Property
Broker	Single Kafka server in the cluster	Stores partitions, serves producer/consumer I/O
Topic	Named category for events	Multi-partition, multi-subscriber (like a DB table)
Partition	Ordered, immutable log within a topic	Unit of parallelism; each record has unique offset
Replica	Copy of a partition on another broker	Leader handles I/O; followers replicate passively
ISR	In-Sync Replica set	Replicas caught up with leader; eligible for election
Controller	Special broker managing metadata	Leader election, partition reassignment, topic creation
Offset	Sequential ID of a record in partition	Monotonically increasing; consumers track position

Data flow: Producer → serialize record → hash key to select partition → send to partition leader broker → leader writes to commit log → followers replicate → consumer polls from leader at its tracked offset.

■ **Follow-up:** 'What happens when a leader broker fails?' Controller detects failure via heartbeat, selects new leader from ISR, broadcasts updated metadata. Clients redirect automatically — typically sub-second failover with KRaft.

Q3. How does Kafka guarantee message ordering? What are the limitations?

Medium Core

Answer:

Kafka guarantees **strict ordering within a single partition only**. Records are assigned monotonically increasing offsets and always read in write order. There is **no ordering across partitions**.

To ensure ordering for a specific entity (e.g., all events for order #123), use a **message key**. Kafka hashes the key (*murmur2(key) % numPartitions*) to deterministically route all records with the same key to the same partition.

```
// All events for the same orderId go to the same partition -> ordered
ProducerRecord<String, String> record =
    new ProducerRecord<>("order-events", orderId, eventPayload);
//                       topic          KEY      value
producer.send(record);

// WARNING: If max.in.flight.requests.per.connection > 1 AND
// enable.idempotence = false, retries can reorder within a partition!
// Solution: enable idempotence (default since Kafka 3.0)
```

■■ **Trap:** 'Does Kafka guarantee global ordering?' NO — only per-partition. If you need total order, use a single partition (sacrificing parallelism). Also: without idempotence enabled, retries can break even per-partition ordering.

Q4. What are acks=0, acks=1, and acks=all? When would you use each?

Medium Producer

Answer:

acks	Behavior	Data Safety	Latency	When to Use

0	No broker ack waited	Possible loss — fire and forget	Lowest	Metrics, logs where loss is acceptable
1	Leader writes to local log then acks	Loss if leader crashes before replication	Low	Moderate safety; tutorial/dev setups
all (-1)	ALL in-sync replicas must ack the write	No data loss when ISR > 1	Highest	Production standard. Orders, payments.

```
Properties props = new Properties();
props.put("acks", "all");                          // Wait for ALL ISR replicas
props.put("min.insync.replicas", "2");             // Broker cfg: at least 2 must ack
props.put("enable.idempotence", "true");           // Prevent duplicate on retry
props.put("retries", Integer.MAX_VALUE);           // Retry indefinitely on failure
// This combination = STRONGEST producer durability guarantee
```

■■ **Trap:** acks=all alone is NOT enough! If *min.insync.replicas=1* (default), 'all' only requires the leader. Always set *min.insync.replicas=2* alongside *acks=all* for true multi-broker durability.

Q5. What is an idempotent producer? How does Kafka prevent duplicate messages on retry?

Hard Producer

Answer:

An idempotent producer ensures **retrying a failed send never creates duplicate records**. Enabled by default since Kafka 3.0 (*enable.idempotence=true*).

Internal Mechanism

- **Producer ID (PID):** Broker assigns a unique PID to each producer session on first send.
- **Sequence Number:** For each *(PID, partition)* pair, producer maintains a monotonically increasing seqNum starting at 0.
- **Broker Dedup:** Every record carries *(PID, partition, seqNum)*. Broker tracks lastCommitted seqNum. If it receives *seqNum <= lastCommitted*, it silently discards the duplicate and returns success.

```
// Scenario: Network timeout during send
// 1. Producer sends (PID=7, partition=3, seq=42) -> broker commits it
// 2. ACK lost in network -> producer thinks it failed
// 3. Producer retries SAME record (PID=7, partition=3, seq=42)
// 4. Broker sees seq=42 already committed -> DISCARD duplicate -> return OK
// Result: Exactly ONE copy in the log. No application-level dedup needed.

// Config (default since Kafka 3.0):
props.put("enable.idempotence", "true");
// Implies: acks=all, retries=MAX_VALUE, max.in.flight.requests <= 5
```

■ **Follow-up:** 'What about across producer restarts?' New PID assigned on restart — idempotence alone can't deduplicate across sessions. For cross-session guarantees, use **transactional.id**. Broker uses it to fence zombie producers via epoch numbers.

Q6. Write a production-grade Kafka producer in Java with acks=all, idempotence, and async callbacks.

Medium Java Code

Answer:

```
import org.apache.kafka.clients.producer.*;
import java.util.Properties;

public class OrderProducer {
    public static void main(String[] args) {
        Properties props = new Properties();
        props.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, "localhost:9092");
        props.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG,
            "org.apache.kafka.common.serialization.StringSerializer");
        props.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG,
            "org.apache.kafka.common.serialization.StringSerializer");

        // -- Durability --
        props.put(ProducerConfig.ACKS_CONFIG, "all");
        props.put(ProducerConfig.ENABLE_IDEMPOTENCE_CONFIG, true);
        props.put(ProducerConfig.RETRIES_CONFIG, Integer.MAX_VALUE);
        props.put(ProducerConfig.MAX_IN_FLIGHT_REQUESTS_PER_CONNECTION, 5);

        // -- Throughput tuning --
        props.put(ProducerConfig.BATCH_SIZE_CONFIG, 32768);       // 32KB
        props.put(ProducerConfig.LINGER_MS_CONFIG, 10);           // 10ms wait
        props.put(ProducerConfig.COMPRESSION_TYPE_CONFIG, "lz4"); // fast compress

        try (KafkaProducer<String, String> producer = new KafkaProducer<>(props)) {
            for (int i = 0; i < 100; i++) {
                String key = "order-" + i;
                String val = "{\"orderId\":" + i + ",\"status\":\"CREATED\"}";

                producer.send(
                    new ProducerRecord<>("order-events", key, val),
                    (metadata, ex) -> {
                        if (ex == null)
                            System.out.printf("Sent P%d offset %d%n",
                                metadata.partition(), metadata.offset());
                        else
                            System.err.println("FAILED: " + ex.getMessage());
                    }
                );
            }
        } // close() flushes all pending batches automatically
    }
}
```

■ **Pro Tip:** Always use **try-with-resources** for KafkaProducer — *close()* calls *flush()* internally, ensuring all pending batches are sent. The callback runs on the I/O thread — keep it fast (no blocking DB calls inside).

Q7. Write a Kafka consumer in Java with manual offset commit and graceful shutdown. Medium Java Code

Answer:

```
import org.apache.kafka.clients.consumer.*;
import java.time.Duration;
import java.util.*;
import java.util.concurrent.atomic.AtomicBoolean;

public class OrderConsumer {
    private static final AtomicBoolean running = new AtomicBoolean(true);

    public static void main(String[] args) {
        Properties props = new Properties();
        props.put(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG, "localhost:9092");
        props.put(ConsumerConfig.GROUP_ID_CONFIG, "order-processing-grp");
        props.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG,
            "org.apache.kafka.common.serialization.StringDeserializer");
        props.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG,
            "org.apache.kafka.common.serialization.StringDeserializer");
        props.put(ConsumerConfig.ENABLE_AUTO_COMMIT_CONFIG, false);   // MANUAL
        props.put(ConsumerConfig.AUTO_OFFSET_RESET_CONFIG, "earliest");
        props.put(ConsumerConfig.MAX_POLL_RECORDS_CONFIG, 100);

        KafkaConsumer<String, String> consumer = new KafkaConsumer<>(props);
        consumer.subscribe(List.of("order-events"));

        Runtime.getRuntime().addShutdownHook(new Thread(() -> {
            running.set(false);
            consumer.wakeup();  // Breaks out of poll() safely
        }));

        try {
            while (running.get()) {
                ConsumerRecords<String, String> records =
                    consumer.poll(Duration.ofMillis(500));
                for (ConsumerRecord<String, String> r : records)
                    processOrder(r);
                if (!records.isEmpty())
                    consumer.commitSync();  // Commit AFTER processing
            }
        } catch (org.apache.kafka.common.errors.WakeupException e) {
            if (running.get()) throw e;
        } finally {
            consumer.commitSync();
            consumer.close();  // Leaves group cleanly -> fast rebalance
        }
    }
    static void processOrder(ConsumerRecord<String, String> r) {
        System.out.printf("key=%s P%d offset=%d%n",
            r.key(), r.partition(), r.offset());
    }
}
```

■■ **Trap:** If you commit offsets BEFORE processing (auto-commit), crashed records are skipped → at-most-once delivery. Always disable auto-commit and call *commitSync()* AFTER processing for at-least-once guarantees.

Q8. What is the difference between session.timeout.ms and max.poll.interval.ms?

Hard Consumer

Answer:

These two configs detect **fundamentally different failure modes** and run on **separate threads**:

Property	session.timeout.ms	max.poll.interval.ms
Default	45,000 ms (45s)	300,000 ms (5 min)
Detects	Dead/crashed consumer (heartbeat thread stops)	Stuck/slow consumer (poll() not called in time)
Mechanism	Background heartbeat thread. No heartbeat in window → broker evicts consumer.	Time between consecutive poll() calls. Exceeded → consumer considered failed & evicted.
Trigger	Process crash, network partition, JVM hang, long GC pause	Processing takes too long, deadlock, slow external service
Tuning	Lower = faster detection but more false positive with GC	Set based on WORST-CASE processing time per batch

```
// Scenario: consumer.poll() returns 500 records
// Processing all 500 takes 6 minutes (slow DB writes)
// max.poll.interval.ms = 300000 (5 min default)
// Result: consumer evicted after 5 min -> REBALANCE triggered
// BUT heartbeat thread is alive! session.timeout is fine!

// FIX options:
props.put("max.poll.interval.ms", "600000");  // Increase to 10 min
props.put("max.poll.records", "50");          // OR reduce batch size
```

■ **Pro Tip:** Interviewers want you to show these are **independent mechanisms on separate threads**. session.timeout = heartbeat thread; max.poll.interval = application thread. A consumer can have a healthy heartbeat but still be evicted for slow processing.

Q9. Explain at-most-once, at-least-once, and exactly-once delivery semantics with Kafka configs.

Medium Delivery

Answer:

Semantic	Producer + Consumer Config	Risk	Use Case
At-Most-Once	Commit offset BEFORE processing. acks=0 or acks=1 + retries=0.	Data loss — missed messages on crash	Metrics, telemetry, non-critical logs
At-Least-Once	Commit offset AFTER processing. acks=all + retries enabled.	Duplicates — message processed 2+ times	Most prod systems + idempotent consumer
Exactly-Once (EOS)	Idempotent producer + Kafka transactions API + consumer with isolation.level=read_committed	None (but +15-20% latency overhead)	Financial txns, cross-topic atomic writes

Practical recommendation: Most production systems use **at-least-once + idempotent consumers**. Make your handlers idempotent (DB upserts, idempotency keys) and you get effective exactly-once without the complexity and latency overhead of the Transactions API.

■ **Follow-up:** 'How do you make a consumer idempotent?' (1) Store processed event IDs in a dedup table, (2) use DB upserts *(INSERT ON CONFLICT UPDATE)*, (3) include Kafka offset in the DB record and check before processing. Strategy depends on your business logic.

Q10. Write a complete Spring Boot Kafka producer and consumer with manual acknowledgment.

Medium Spring Boot

Answer:

application.yml

```
spring:
  kafka:
    bootstrap-servers: localhost:9092
    producer:
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer: org.apache.kafka.common.serialization.StringSerializer
      acks: all
      retries: 2147483647
      properties:
        enable.idempotence: true
        linger.ms: 10
        compression.type: lz4
    consumer:
      group-id: order-group
      auto-offset-reset: earliest
      enable-auto-commit: false
      key-deserializer: org.apache.kafka.common.serialization.StringDeserializer
      value-deserializer: org.apache.kafka.common.serialization.StringDeserializer
      properties:
        max.poll.records: 100
    listener:
      concurrency: 3
      ack-mode: manual
```

Producer Service

```
@Service @RequiredArgsConstructor @Slf4j
public class OrderProducer {
    private final KafkaTemplate<String, String> kafkaTemplate;

    public void publish(String orderId, String payload) {
        kafkaTemplate.send("order-events", orderId, payload)
            .whenComplete((result, ex) -> {
                if (ex == null)
                    log.info("Sent {} to P{} offset {}", orderId,
                        result.getRecordMetadata().partition(),
                        result.getRecordMetadata().offset());
                else
                    log.error("Failed {}: {}", orderId, ex.getMessage());
            });
    }
}
```

Consumer with Manual Ack + Header Injection

```
@Service @Slf4j
public class OrderConsumer {

    @KafkaListener(topics = "order-events", groupId = "order-group")
    public void consume(
            @Payload String message,
            @Header(KafkaHeaders.RECEIVED_KEY) String key,
            @Header(KafkaHeaders.RECEIVED_PARTITION) int partition,
            @Header(KafkaHeaders.OFFSET) long offset,
            Acknowledgment ack) {

        log.info("Received key={} P{} offset={}", key, partition, offset);
        orderService.process(message);   // Business logic
        ack.acknowledge();               // Commit offset AFTER success
    }
}
```

■ **Pro Tip:** Key configs: *ack-mode: manual* gives you explicit control over offset commits, *concurrency: 3* creates 3 consumer threads (each gets partitions), and *enable-auto-commit: false* lets Spring manage commits via the Acknowledgment object.

Q11. How does @KafkaListener work internally in Spring Boot? Explain the full lifecycle.

Hard Spring Boot

Answer:

When the Spring context starts, a multi-step chain creates the listener infrastructure:

- **1. Scanning:** *KafkaListenerAnnotationBeanPostProcessor* scans all beans for *@KafkaListener* annotations.
- **2. Registration:** For each annotated method, creates *MethodKafkaListenerEndpoint* and registers with *KafkaListenerEndpointRegistry*.
- **3. Container creation:** Registry uses *ConcurrentKafkaListenerContainerFactory* to create *ConcurrentMessageListenerContainer*.
- **4. Thread allocation:** Container creates N *KafkaMessageListenerContainer* instances (N = concurrency), each with its own *KafkaConsumer* and dedicated poll thread.
- **5. Poll loop:** Each thread runs: *poll() → deserialize → dispatch to annotated method → handle ack → error handler → repeat*.
- **6. Lifecycle:** Containers implement *SmartLifecycle* — auto-start after context init, graceful stop on shutdown.

```
// Programmatic control via the registry:
@Autowired KafkaListenerEndpointRegistry registry;

registry.getListenerContainer("orderId").stop();    // Stop listener
registry.getListenerContainer("orderId").pause();   // Pause (heartbeat alive, no records)
registry.getListenerContainer("orderId").resume();  // Resume processing

// @KafkaListener with explicit ID for registry lookup:
@KafkaListener(id = "orderId", topics = "order-events")
public void handle(String msg, Acknowledgment ack) {
    orderService.process(msg);
    ack.acknowledge();
}
```

■ **Pro Tip:** Mentioning the full chain (BeanPostProcessor → Registry → ContainerFactory → Container → poll loop) shows deep Spring Kafka mastery. Most candidates only know 'it creates a consumer' — the lifecycle differentiates senior engineers.

Q12. How does Kafka achieve millions of messages per second? Explain the key design decisions.

Medium Architecture

Answer:

- **Sequential I/O:** All writes are sequential appends. Sequential disk I/O is 100–1000x faster than random I/O — often approaching memory speed on modern SSDs.
- **OS Page Cache:** Kafka delegates caching to the OS. Frequently read data served from RAM without JVM heap overhead or GC pauses.
- **Zero-Copy (sendfile):** Data transfers directly from page cache → NIC socket via *sendfile()* syscall. Eliminates 2 CPU memory copies and 2 context switches.
- **Batching:** Producers batch multiple records per request. Consumers fetch in bulk. Amortizes network round-trips and disk seeks.
- **Compression:** Entire batches compressed (lz4/zstd/snappy). 50–80% reduction in network bandwidth and disk I/O.
- **Partitioning:** Load distributed across brokers and disks. Each partition on a separate disk enables parallel I/O.

Step	Traditional Transfer (4 copies)	Kafka Zero-Copy (2 copies)
1	Disk → Kernel read buffer (DMA)	Disk → Page cache (DMA)
2	Kernel → App buffer (CPU copy)	Page cache → NIC via sendfile (DMA)
3	App → Socket buffer (CPU copy)	— eliminated —
4	Socket → NIC (DMA)	— eliminated —
CPU copies	2 (expensive context switches)	0 (no user-space involvement)

■ **Follow-up:** 'Why is Kafka's heap only 4–6GB?' Data lives in OS page cache, NOT JVM heap. Larger heap wastes RAM the page cache could use. More page cache = more 'hot' data served from memory = faster consumer reads.

Q13. What is the Kafka Transactions API? Write a transactional produce-consume-produce example.

Hard Transactions

Answer:

Kafka Transactions enable **atomic writes across multiple partitions and topics**. Either all records commit or all abort. Consumers with *isolation.level=read_committed* never see partial or aborted data.

```
Properties props = new Properties();
props.put("bootstrap.servers", "localhost:9092");
props.put("transactional.id", "order-txn-1");  // Stable across restarts
props.put("enable.idempotence", "true");        // Required for transactions
props.put("acks", "all");
props.put("key.serializer", "...StringSerializer");
props.put("value.serializer", "...StringSerializer");

KafkaProducer<String, String> producer = new KafkaProducer<>(props);
producer.initTransactions();  // Register with transaction coordinator

try {
    producer.beginTransaction();

    // Atomic: BOTH sends commit or BOTH abort
    producer.send(new ProducerRecord<>("orders", "k1", "order-created"));
    producer.send(new ProducerRecord<>("audit-log", "k1", "audit-entry"));

    // Include consumer offset in SAME transaction (read-process-write EOS)
    Map<TopicPartition, OffsetAndMetadata> offsets = Map.of(
        new TopicPartition("input-topic", 0),
        new OffsetAndMetadata(lastOffset + 1));
    producer.sendOffsetsToTransaction(offsets, consumerGroupMetadata);

    producer.commitTransaction();      // Atomic commit: records + offset
} catch (ProducerFencedException e) {
    producer.close();                  // Zombie fenced - fatal, cannot recover
} catch (KafkaException e) {
    producer.abortTransaction();       // Rollback - records become invisible
}
```

■ **Follow-up:** 'What is epoch fencing?' Each *transactional.id* has an epoch (version). On *initTransactions()*, epoch increments. If a zombie producer with stale epoch tries to write, broker rejects with *ProducerFencedException* — prevents split-brain duplicates.

Q14. How do you handle a poison pill message in Kafka? Explain a production strategy.

Hard Error Handling

Answer:

A **poison pill** is a message that ALWAYS fails — corrupted data, undeserializable bytes, or invalid business content. Without handling, it blocks the partition forever.

5-Layer Defense Strategy

- **L1 — Deserialization:** *ErrorHandlingDeserializer* catches deser failures. Record passed to error handler instead of crashing consumer.
- **L2 — Validation:** Validate business content right after deserialization. Reject invalid records early.
- **L3 — Retry:** *DefaultErrorHandler* with exponential backoff for transient failures. *addNotRetryableExceptions()* for permanent failures.
- **L4 — Dead Letter Topic:** After retry exhaustion, send to DLT with metadata (original topic, partition, offset, exception).
- **L5 — Monitoring:** Alert on DLT lag > 0. Build inspection/replay dashboard.

```
// Spring Boot: Production poison pill handling
@Bean
public DefaultErrorHandler errorHandler(KafkaTemplate<?, ?> tpl) {
    DeadLetterPublishingRecoverer recoverer =
        new DeadLetterPublishingRecoverer(tpl,
            (rec, ex) -> new TopicPartition(rec.topic() + "-dlt", rec.partition()));

    ExponentialBackOff backOff = new ExponentialBackOff(1000L, 2.0);
    backOff.setMaxElapsedTime(10000L);  // 1s -> 2s -> 4s -> DLT

    DefaultErrorHandler handler = new DefaultErrorHandler(recoverer, backOff);

    // Permanent failures -> skip retry, go DIRECTLY to DLT
    handler.addNotRetryableExceptions(
        DeserializationException.class,     // Bad bytes
        ClassCastException.class,           // Wrong type
        NullPointerException.class,         // Null required field
        IllegalArgumentException.class);    // Invalid business data
    return handler;
}
```

■■ **Trap:** Without *addNotRetryableExceptions()*, a *DeserializationException* is retried 3 times (wasting 7 seconds) before DLT. Deser errors are NEVER transient — send them to DLT immediately.

Q15. What is KRaft? Why is Kafka removing ZooKeeper and what are the benefits?

Medium Architecture

Answer:

KRaft (Kafka Raft) is Kafka's built-in consensus protocol replacing ZooKeeper for cluster metadata management. It uses the Raft algorithm to elect controllers and manage metadata natively within Kafka brokers.

Aspect	ZooKeeper Mode	KRaft Mode

Metadata store	External ZooKeeper ensemble (3-5 separate nodes to deploy)	Internal __cluster_metadata topic (no external dependency at all)
Controller election	ZK ephemeral znodes	Raft-based quorum voting
Partition limit	~200K per cluster (ZK bottleneck)	Millions (metadata is a Kafka topic)
Failover speed	Minutes (full state read from ZK)	Seconds (Raft leader election)
Ops complexity	Two systems to deploy, monitor, upgrade, and secure separately	Single system to manage end-to-end
Status (2024+)	Deprecated; removal planned	Production-ready since Kafka 3.3+

```
# KRaft server.properties (no ZooKeeper required!)
process.roles=broker,controller
node.id=1
controller.quorum.voters=1@broker1:9093,2@broker2:9093,3@broker3:9093
controller.listener.names=CONTROLLER
listeners=PLAINTEXT://broker1:9092,CONTROLLER://broker1:9093
log.dirs=/var/kafka/data
# Notice: NO zookeeper.connect property!
```

■ **Pro Tip:** In interviews: 'We migrated to KRaft because it simplifies operations, improves failover from minutes to seconds, removes the ~200K partition ceiling, and eliminates a separate ZooKeeper cluster to manage.' This signals production experience.

Q16. How do consumer group rebalances work? Compare Eager vs Cooperative rebalancing.

Hard Consumer

Answer:

A **rebalance** occurs when consumer group membership changes — a consumer joins, leaves, crashes, or topic partitions change. The group coordinator redistributes partitions. Two protocols exist with vastly different production impact:

Strategy	Type	Behavior	Production Impact
RangeAssignor (old default)	Eager	ALL partitions revoked from ALL consumers, then reassigned	Stop-the-world pause. No processing during entire rebalance.
RoundRobin Assignor	Eager	Even distribution across all consumers, still full revocation	Stop-the-world. Better balance than Range.
StickyAssignor	Eager	Minimizes partition movement; consumers keep current partitions	Stop-the-world but minimal reassignment.
CooperativeSticky Assignor	Cooperative (Incremental)	ONLY affected partitions revoked. Other consumers continue processing uninterrupted.	No stop-the-world! Recommended for all production clusters.

```
// Production config: CooperativeStickyAssignor
props.put(ConsumerConfig.PARTITION_ASSIGNMENT_STRATEGY_CONFIG,
    "org.apache.kafka.clients.consumer.CooperativeStickyAssignor");

// Spring Boot application.yml
spring:
  kafka:
    consumer:
      properties:
        partition.assignment.strategy: \
          org.apache.kafka.clients.consumer.CooperativeStickyAssignor
        group.instance.id: ${HOSTNAME}    # Static membership (K8s)
        session.timeout.ms: 45000
```

■■ **Trap:** During EAGER rebalancing, ALL consumption stops — even partitions NOT changing owners. A 3-pod rolling deployment with 12 partitions triggers 3 full stop-the-world rebalances. CooperativeStickyAssignor eliminates this — only affected partitions are revoked.

Q17. What is the difference between Kafka Connect and Kafka Streams? When would you use each?

Easy Ecosystem

Answer:

Aspect	Kafka Connect	Kafka Streams
Purpose	Move data IN/OUT of Kafka (integration framework)	Process/transform data ALREADY in Kafka (library)

How it runs	Standalone or distributed workers running source/sink connectors	Embedded in your Java/Spring Boot application as a library
Coding	Config-only (JSON). No code for standard connectors.	Java/Scala using DSL or Processor API
State	Stateless — just moves data	Stateful — local state stores, KTable, windowed aggregations
Examples	Debezium (CDC from PostgreSQL), Elasticsearch Sink, S3 Sink	Real-time fraud detection, stream-table joins, windowed aggregations
Scaling	Add more worker nodes to distribute connector tasks	Add more app instances; state is repartitioned automatically

```
// Kafka Connect: Debezium CDC (zero code, JSON config)
{ "name": "postgres-cdc",
  "config": {
    "connector.class": "io.debezium.connector.postgresql.PostgresConnector",
    "database.hostname": "postgres",
    "database.dbname": "orders",
    "table.include.list": "public.orders",
    "topic.prefix": "cdc"
  }
}

// Kafka Streams: Real-time processing (Java)
KStream<String, OrderEvent> orders = builder.stream("order-events");
orders.filter((key, val) -> val.getAmount() > 1000)
      .groupByKey()
      .windowedBy(TimeWindows.ofSizeWithNoGrace(Duration.ofMinutes(5)))
      .count()
      .toStream()
      .to("high-value-order-counts");
```

■ **Follow-up:** 'When Streams vs Flink/Spark?' Use Streams for moderate complexity (filtering, joins, windowed aggregations) embedded in your microservice — no cluster needed. Use Flink/Spark for complex event processing, ML pipelines, or exactly-once across heterogeneous sinks.

■ This completes **Part 1 (Questions 1–17)** covering Kafka Fundamentals, Architecture, Producers, Consumers, Java API, Spring Boot, Transactions, Error Handling, KRaft, Rebalancing, and Ecosystem (Connect/Streams). **Part 2** will cover Questions 18–34: Avro & Schema Registry, Schema Evolution, Performance Tuning, Exactly-Once, and Design Patterns. Say **'next'** to continue!

100 Kafka Interview

Questions & Answers

with Detailed Code Examples

Part 2: Questions 18 – 34

Avro • Schema Registry • Schema Evolution • Performance Tuning

Outbox Pattern • Backpressure • Batch Listeners • DLT Replay

by **TechTribers**

@techtribes • www.techtriber.com

Q18. What is Apache Avro and why is it the preferred serialization format for Kafka?

Medium Avro

Answer:

Apache Avro is a **binary serialization framework** that stores the schema separately from the data. Unlike JSON (schema embedded in every record) or Protobuf (requires .proto compilation), Avro achieves compact payloads with runtime schema resolution.

Dimension	JSON	Protobuf	Avro
Payload size	Large (field names in every record)	Small (field tags as integers)	Smallest (no field names in payload)
Schema in payload	Yes (self-describing)	No (need .proto file)	No (schema ID in 5-byte header)
Schema evolution	No enforcement; breaks silently	Good; backward/ forward compat	Best; formal rules + Schema Registry
Code generation	Not needed	Required (protoc)	Optional (can use GenericRecord)
Kafka ecosystem	Works but no governance	Supported by Schema Registry	Native support; first-class citizen
Human readable	Yes (text)	No (binary)	No (binary)

■ **Pro Tip:** Key interview phrase: *'Avro gives us 50-70% smaller payloads than JSON, schema evolution with formal compatibility rules, and centralized governance via Schema Registry — all without requiring code generation.'*

Q19. What is the Confluent Schema Registry? Explain the producer and consumer flow.

Medium Schema Registry

Answer:

The Schema Registry is a **centralized service** that stores, versions, and enforces compatibility rules for schemas. It prevents breaking changes from reaching production.

Producer Flow (Serialization)

- **1.** Producer serializes record using Avro schema.
- **2.** KafkaAvroSerializer checks local cache for schema ID.
- **3.** Cache miss → POST schema to Registry. Registry runs compatibility check against previous versions.
- **4.** If compatible → returns schema ID. If NOT compatible → 409 Conflict, record rejected.
- **5.** Serializer prepends **5-byte header** (magic byte 0x00 + 4-byte schema ID) to Avro binary payload.

Consumer Flow (Deserialization)

- **1.** Consumer reads record bytes.
- **2.** KafkaAvroDeserializer extracts schema ID from 5-byte header.
- **3.** Fetches **writer schema** from Registry (cached after first fetch).
- **4.** Performs **Avro schema resolution**: writer schema vs reader schema (consumer's local schema).
- **5.** Returns deserialized object (SpecificRecord or GenericRecord).

```
// Wire format of every Avro-serialized Kafka record:
// | Byte 0  | Bytes 1-4           | Bytes 5+            |
// |---------|---------------------|---------------------|
// | 0x00    | Schema ID (4 bytes, | Avro binary payload |
// | (magic) | big-endian int)     |                     |

// Key REST API endpoints:
// POST   /subjects/{subject}/versions       Register new schema
// GET    /schemas/ids/{id}                  Fetch schema by ID
// POST   /compatibility/subjects/{s}/versions/latest  Check compat
// PUT    /config/{subject}                  Set compatibility level
```

■ **Follow-up:** 'How does the consumer know which schema to use?' The schema ID is embedded in every record. Consumer fetches the writer schema by ID, then uses Avro's schema resolution rules to handle differences between writer and reader schemas automatically.

Q20. Explain Avro schema evolution. What changes are backward, forward, and fully compatible?

Hard Schema Registry

Answer:

Schema Change	Backward?	Forward?	Full?	Why
Add field WITH default	Yes	Yes	Yes	Old consumers use default; new consumers read new field
Add field WITHOUT default	No	Yes	No	Old consumer can't provide missing field value

Remove field that had default	Yes	Yes	Yes	Both handle missing field via default value
Remove field without default	Yes	No	No	New consumer ignores it; old consumer expects it
Change type (int → long)	No	No	No	Only Avro-promoted types work (int→long OK)
Rename field	No	No	No	Avro matches by name; different name = break
Add enum symbol	Yes	No	No	Old consumer doesn't know the new enum value

```
{
  "type": "record",
  "name": "OrderEvent",
  "namespace": "com.techtribers.avro",
  "fields": [
    { "name": "orderId", "type": "string" },
    { "name": "amount",  "type": "double" },
    { "name": "status",  "type": "string" },
    // v2: Added WITH default -> BACKWARD + FORWARD compatible
    { "name": "loyaltyTier", "type": "string", "default": "STANDARD" },
    // v2: Nullable with null default -> BACKWARD + FORWARD compatible
    { "name": "promoCode", "type": ["null", "string"], "default": null }
  ]
}
```

■■ **Trap:** 'Can I rename a field in Avro?' NO — Avro matches fields by name. Renaming is a breaking change. You can use *aliases* to map old names but it's fragile. Best practice: add new field, deprecate old one.

Q21. Write a Spring Boot Kafka producer and consumer using Avro + Schema Registry.

Hard Spring Boot + Avro

Answer:

1. Avro Schema (src/main/avro/OrderEvent.avsc)

```
{ "type": "record", "name": "OrderEvent",
  "namespace": "com.techtribers.kafka.avro",
  "fields": [
    { "name": "orderId",   "type": "string" },
    { "name": "userId",    "type": "string" },
    { "name": "amount",    "type": "double" },
    { "name": "status",    "type": "string" },
    { "name": "createdAt", "type": {"type":"long","logicalType":"timestamp-millis"} }
]}
```

2. application.yml

```
spring:
  kafka:
    bootstrap-servers: localhost:9092
    properties:
      schema.registry.url: http://localhost:8081
    producer:
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer: io.confluent.kafka.serializers.KafkaAvroSerializer
      acks: all
    consumer:
      group-id: order-avro-group
      auto-offset-reset: earliest
      enable-auto-commit: false
      key-deserializer: org.apache.kafka.common.serialization.StringDeserializer
      value-deserializer: io.confluent.kafka.serializers.KafkaAvroDeserializer
      properties:
        specific.avro.reader: true   # Returns SpecificRecord, not GenericRecord
    listener:
      ack-mode: manual
```

3. Producer + Consumer

```
// Producer — type-safe with generated Avro class
@Service @RequiredArgsConstructor @Slf4j
public class OrderAvroProducer {
    private final KafkaTemplate<String, OrderEvent> kafkaTemplate;

    public void publish(String orderId, String userId, double amount) {
        OrderEvent event = OrderEvent.newBuilder()
            .setOrderId(orderId).setUserId(userId).setAmount(amount)
            .setStatus("CREATED").setCreatedAt(Instant.now().toEpochMilli())
            .build();
        kafkaTemplate.send("order-events", orderId, event)
            .whenComplete((r, ex) -> {
                if (ex == null) log.info("Avro sent {} P{}", orderId,
                    r.getRecordMetadata().partition());
                else log.error("Avro fail {}: {}", orderId, ex.getMessage());
            });
    }
}

// Consumer — Avro deserialized to SpecificRecord automatically
@Service @Slf4j
public class OrderAvroConsumer {
    @KafkaListener(topics = "order-events", groupId = "order-avro-group")
    public void consume(@Payload OrderEvent event,
                        @Header(KafkaHeaders.RECEIVED_PARTITION) int p,
                        @Header(KafkaHeaders.OFFSET) long offset,
                        Acknowledgment ack) {
        log.info("Avro: id={} amt={} P{} off={}", event.getOrderId(),
            event.getAmount(), p, offset);
        orderService.processOrder(event);
        ack.acknowledge();
    }
}
```

■ **Pro Tip:** Set *specific.avro.reader=true* to get type-safe generated classes. Without it, you receive *GenericRecord* and must access fields by string name: *record.get("orderId")* — no compile-time safety.

Q22. What is the difference between SpecificRecord and GenericRecord in Avro?

Medium Avro

Answer:

Aspect	SpecificRecord	GenericRecord
What it is	Generated Java class from .avsc schema (via avro-maven-plugin)	Dynamic record; fields accessed by string name at runtime
Type safety	Full compile-time type checking. event.getOrderId() returns String	No compile-time checks. event.get("orderId") returns Object
IDE support	Full autocomplete, refactoring	None — string-based field access
Schema coupling	Tightly coupled; schema changes require regeneration + redeploy	Loosely coupled; adapts to any schema at runtime
Config	specific.avro.reader=true	specific.avro.reader=false (default)
Best for	Core business services where type safety matters	Schema-agnostic services: audit logger, data pipeline, CDC relay

```
// SpecificRecord — type-safe, generated class
OrderEvent event = OrderEvent.newBuilder()
    .setOrderId("ORD-123")    // Compile error if wrong type
    .setAmount(99.50)
    .build();
String id = event.getOrderId();  // Returns String, not Object

// GenericRecord — dynamic, no code generation needed
GenericRecord record = new GenericData.Record(schema);
record.put("orderId", "ORD-123");
record.put("amount", 99.50);
Object id = record.get("orderId");  // Returns Object, must cast
```

■ **Follow-up:** 'When would you use GenericRecord in production?' For cross-cutting services that process ANY schema: audit loggers, DLT consumers, data lake ingestion, schema migration tools. These services shouldn't be redeployed when upstream schemas change.

Q23. What is the difference between linger.ms and batch.size? How do they affect throughput?

Medium Producer Tuning

Answer:

Config	Default	What It Does	Effect

batch.size	16384 (16KB)	Max bytes per batch for a single partition. Batch sent when full.	Larger → more records/request → higher throughput but more memory usage
linger.ms	0	How long to wait for more records before sending incomplete batch.	Higher → fuller batches → higher throughput but added latency

Interaction: A batch is sent when EITHER batch.size is reached OR linger.ms elapses — whichever comes first. With *linger.ms=0* (default), the producer sends on the next I/O cycle, resulting in many tiny batches. Setting *linger.ms=5-20* allows time for records to accumulate.

```
// LOW LATENCY (sacrifice throughput)
props.put("batch.size", 16384);    // 16KB default
props.put("linger.ms", 0);         // Send immediately

// HIGH THROUGHPUT (add small latency)
props.put("batch.size", 65536);    // 64KB batches
props.put("linger.ms", 20);        // Wait 20ms to fill batch
props.put("compression.type", "lz4");  // Compress full batches
// Result: ~5-10x fewer network requests
```

■ **Follow-up:** 'When is linger.ms=0 with large batch.size effective?' When the producer sends faster than the network can drain — batches fill naturally without delay. linger.ms only helps when production rate is low or bursty.

Q24. What is the Outbox Pattern? Why is it needed with Kafka and how do you implement it?

Hard Design Patterns

Answer:

The Outbox Pattern solves: **how to atomically update a database AND publish an event to Kafka**. These are two independent systems — no single transaction spans both.

The Dual-Write Problem

- **Failure 1:** DB commits, Kafka send fails → order exists but no event (downstream unaware).
- **Failure 2:** Kafka send succeeds, DB rolls back → event published but order doesn't exist.

Solution: Single DB Transaction + CDC

- **Step 1:** In ONE database transaction: insert business data AND insert event into *outbox* table.
- **Step 2:** CDC tool (Debezium) reads database WAL and publishes outbox rows to Kafka.
- **Guarantee:** Event published if and only if DB transaction committed. Atomic.

```
@Transactional  // Single DB transaction — both writes atomic
public Order createOrder(CreateOrderRequest req) {
    Order order = orderRepo.save(Order.from(req));

    outboxRepo.save(OutboxEvent.builder()   // Same transaction!
        .aggregateId(order.getId().toString())
        .aggregateType("Order")
        .eventType("OrderCreated")
        .payload(objectMapper.writeValueAsString(order))
        .build());

    return order;
    // If exception -> both order AND outbox rolled back
    // If success -> Debezium picks up outbox row via CDC -> publishes to Kafka
}

-- Outbox table schema:
-- CREATE TABLE outbox (
--   id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
--   aggregate_type VARCHAR(100) NOT NULL,
--   aggregate_id   VARCHAR(100) NOT NULL,
--   event_type     VARCHAR(100) NOT NULL,
--   payload        JSONB NOT NULL,
--   created_at     TIMESTAMP DEFAULT now()
-- );
```

■ **Follow-up:** 'Can Kafka transactions replace Outbox?' No — Kafka transactions are Kafka-to-Kafka only. They cannot wrap a database write. Outbox bridges DB and Kafka atomically. Use *@Transactional* for DB + Debezium CDC for Kafka.

Q25. How does Kafka handle backpressure? What happens when consumers are slower than producers?

Medium Core

Answer:

Kafka handles backpressure differently from push-based systems because consumers **pull** data at their own pace. A slow consumer simply falls behind — its consumer lag grows.

Backpressure Chain

- **Producer side:** If broker can't keep up, producer's *buffer.memory* fills. When full, *send()* blocks up to *max.block.ms* (60s), then throws TimeoutException.
- **Broker side:** Handles load well (sequential I/O). Real limits: disk space, network bandwidth, replica lag.
- **Consumer side:** Lag grows. If consumer falls behind more than retention period → **data loss** (records deleted before consumption).

Mitigation Strategies

- **Scale consumers:** Add instances (up to partition count).
- **Batch listeners:** Use *setBatchListener(true)* + bulk DB inserts for 5-10x throughput.
- **Increase partitions:** More parallelism (but can't decrease later!).
- **Increase retention:** Buy time with longer *log.retention.hours*.
- **Pause/resume:** Spring Kafka *container.pause()* stops fetching while processing catches up.

■■ **Trap:** Unlike RabbitMQ (QoS prefetch-based backpressure), Kafka has NO built-in consumer backpressure. The consumer just falls behind silently. If you're not monitoring consumer lag, you're blind to backpressure.

Q26. What is the difference between @RetryableTopic (non-blocking) and DefaultErrorHandler (blocking) retry?

Hard Spring Boot

Answer:

Aspect	DefaultErrorHandler (Blocking Retry)	@RetryableTopic (Non-Blocking Retry)
Mechanism	Thread.sleep() inside poll loop on same consumer thread	Failed record forwarded to retry topic; separate listener
Impact on other records	ALL records in partition BLOCKED until retry done	Other records processed immediately — unblocked
Ordering	PRESERVED — retry at same offset position	LOST — record moves to different topic/partition
Long backoffs (30s, 1min)	DANGEROUS — blocks thread, may exceed max.poll.interval	SAFE — consumer continues; retry topic handles delay
Extra topics	None needed	Auto-creates: {topic}-retry-0, {topic}-retry-1, {topic}-dlt
Best for	Short retries (< 10s total), ordering is critical	External service calls, long retries, high throughput

```
@RetryableTopic(
    attempts = "4",              // 1 original + 3 retries
    backoff = @Backoff(delay = 5000, multiplier = 3.0, maxDelay = 60000),
    topicSuffixingStrategy = TopicSuffixingStrategy.SUFFIX_WITH_INDEX_VALUE,
    include = { TransientDataAccessException.class, TimeoutException.class },
    exclude = { DeserializationException.class }
)
@KafkaListener(topics = "payment-events", groupId = "payment-grp")
public void process(PaymentEvent event) {
    paymentGateway.charge(event);   // May throw TransientException
}

@DltHandler
public void handleDlt(PaymentEvent event,
        @Header(KafkaHeaders.EXCEPTION_MESSAGE) String err) {
    log.error("Payment permanently failed: {}", event.getId());
    refundService.initiateRefund(event);
}
// Flow: payment-events -> retry-0 (5s) -> retry-1 (15s) -> retry-2 (45s) -> dlt
```

■ **Pro Tip:** Interview gold: 'Blocking when ordering matters and retries are short. Non-blocking when calling external APIs where failures may last minutes — because blocking would trigger max.poll.interval.ms expiry and cause a rebalance.'

Q27. Write a Spring Boot batch listener with manual ack and bulk database insert.

Medium Spring Boot

Answer:

```
// 1. Batch Listener Container Factory
@Bean
public ConcurrentKafkaListenerContainerFactory<String, String>
        batchFactory(ConsumerFactory<String, String> cf) {
    var factory = new ConcurrentKafkaListenerContainerFactory<String, String>();
    factory.setConsumerFactory(cf);
    factory.setBatchListener(true);          // Enable batch mode
    factory.setConcurrency(6);
    factory.getContainerProperties().setAckMode(
        ContainerProperties.AckMode.MANUAL); // Manual ack for entire batch
    factory.setCommonErrorHandler(
        new DefaultErrorHandler(new FixedBackOff(2000L, 3)));
    return factory;
}

// 2. Batch Listener — receives List instead of single record
@KafkaListener(topics = "sensor-data", groupId = "sensor-batch",
               containerFactory = "batchFactory")
public void consumeBatch(
        List<ConsumerRecord<String, String>> records,
        Acknowledgment ack) {

    log.info("Batch: {} records", records.size());

    List<SensorEntity> entities = records.stream()
        .map(r -> SensorEntity.builder()
            .sensorId(r.key())
            .reading(Double.parseDouble(r.value()))
            .partition(r.partition())
            .kafkaOffset(r.offset())
            .build())
        .toList();

    try {
        sensorRepo.saveAll(entities);  // Bulk DB insert (single query)
        ack.acknowledge();              // Commit offsets AFTER success
    } catch (DataAccessException e) {
        log.error("Batch save failed", e);
        throw e;  // Triggers error handler -> retry entire batch
    }
}
```

■ **Pro Tip:** Batch listeners give 5-10x throughput improvement over record listeners. Key: *saveAll()* for bulk inserts, MANUAL ack after batch succeeds, and *fetch.min.bytes=65536* in consumer config for fuller batches.

Q28. What is log compaction? How is it different from retention-based deletion?

Medium Internals

Answer:

Aspect	Retention-Based Deletion	Log Compaction
Config	cleanup.policy=delete (default)	cleanup.policy=compact
What is removed	Entire segments older than retention time/size limit	Older records with duplicate keys (keeps latest per key)
Guarantee	Data older than retention is deleted permanently	Latest value for every key is ALWAYS available
Use case	Time-series data, event logs, transient messages	Stateful data: user profiles, config, CDC changelogs
Null value	No special meaning	Tombstone — signals key deletion on next compaction
Combination	cleanup.policy=delete,compact	Compaction first, then retention applied to compacted data

Log compaction is how Kafka acts as a **database for latest state**. Kafka Streams KTable internally uses compacted topics. The __consumer_offsets topic is compacted (latest offset per group/partition always available).

■ **Follow-up:** 'How do you delete a key from a compacted topic?' Send a record with the same key and **null value** (tombstone). During the next compaction cycle, the key and all its previous values are removed.

Q29. How do you tune Kafka producers for maximum throughput vs minimum latency?

Medium Performance

Answer:

Config	Max Throughput	Min Latency	Why
acks	1 (or all for safety)	0	all waits for ISR; 0 is fire-and-forget

batch.size	65536 (64KB)	0 (no batching)	Larger batches = fewer requests = more throughput
linger.ms	20-50	0	Delay to fill batches; 0 = send immediately
compression	zstd	none	Best ratio but CPU cost; none = zero compress time
buffer.memory	128MB	32MB (default)	More buffer = handle bursts without blocking
max.in.flight	5	1	5 allows pipelining; 1 = strict ordering

```
// BALANCED PRODUCTION CONFIG (most systems):
props.put("acks", "all");               // Safety first
props.put("batch.size", 32768);         // 32KB batches
props.put("linger.ms", 10);             // Small delay for batching
props.put("compression.type", "lz4");   // Fast compression
props.put("enable.idempotence", "true");// Safe with 5 in-flight
// Good throughput + strong durability + acceptable latency
```

■ **Pro Tip:** Most production systems choose a middle ground: acks=all (safety), linger.ms=5-10 (small delay), batch.size=32KB, compression=lz4 (fast). This gives strong durability with good throughput. Only sacrifice safety for truly non-critical data.

Q30. What is a Dead Letter Topic (DLT)? How do you build a replay mechanism?

Hard Error Handling

Answer:

A DLT is a special topic where permanently failed records are sent after retry exhaustion. It prevents poison pills from blocking consumers and provides an audit trail for investigation.

DLT Replay REST API

```
@RestController @RequestMapping("/api/dlt") @RequiredArgsConstructor
public class DltReplayController {
    private final KafkaTemplate<String, String> kafkaTemplate;
    private final DltRepository dltRepo;

    @PostMapping("/replay/{id}")
    public ResponseEntity<?> replay(@PathVariable Long id) {
        DltRecord rec = dltRepo.findById(id)
            .orElseThrow(() -> new NotFoundException("Not found"));

        kafkaTemplate.send(rec.getOriginalTopic(), rec.getKey(), rec.getPayload())
            .whenComplete((result, ex) -> {
                if (ex == null) {
                    rec.setStatus("REPLAYED");
                    rec.setReplayedAt(Instant.now());
                    dltRepo.save(rec);
                }
            });
        return ResponseEntity.accepted().body("Replay initiated");
    }

    @PostMapping("/replay-all")
    public ResponseEntity<?> replayAll(@RequestParam String topic,
                                       @RequestParam(defaultValue="100") int limit) {
        List<DltRecord> records = dltRepo
            .findByOriginalTopicAndStatus(topic, "PENDING", limit);
        records.forEach(r ->
            kafkaTemplate.send(r.getOriginalTopic(), r.getKey(), r.getPayload()));
        return ResponseEntity.ok(records.size() + " records replayed");
    }
}
```

DLT best practices: (1) Store in DB with metadata (topic, error, timestamp, retry count). (2) Build ops dashboard for inspection. (3) Alert when DLT count > 0. (4) Set 30+ day retention on DLT topics. (5) Track replay success rate as a metric.

Q31. How does Kafka store data on disk? Explain segments, indexes, and the retention mechanism.

Medium Internals

Answer:

Each partition is a directory. Inside, the log is divided into **segments**:

File	Extension	Purpose
Log file	.log	Actual records. Append-only, immutable once segment rolls.
Offset index	.index	Sparse offset → file position map. Memory-mapped for fast lookups.
Time index	.timeindex	Timestamp → offset map. Used for time-based seeks (offsetsForTimes).

Leader epoch	.leader-epoch-checkpoint	Tracks leader epoch per partition. Used in failover recovery.
Snapshot	.snapshot	Producer sequence numbers for idempotent dedup across restarts.

```
# Partition directory on disk:
/var/kafka/data/order-events-0/
  00000000000000000000.log         # Segment: offsets 0-49999
  00000000000000000000.index       # Sparse offset index
  00000000000000000000.timeindex   # Time-based index
  00000000000000050000.log         # Segment: offsets 50000+
  00000000000000050000.index
  leader-epoch-checkpoint

# Retention: deletes at SEGMENT level (not record level)
# Active segment (receiving writes) is NEVER deleted
# log.retention.hours=168 (7 days default)
# log.segment.bytes=1073741824 (1GB per segment)
```

■ **Follow-up:** 'Why sparse index and not dense index?' Dense index would require one entry per record — too much memory. Sparse index stores every Nth offset (default 4KB intervals). Kafka binary-searches the sparse index, then does a sequential scan in the .log file for the exact offset. Fast enough for Kafka's access patterns.

Q32. What is consumer lag and how do you monitor it in production?

Medium Operations

Answer:

Consumer lag = Log-End Offset (latest) – Last Committed Offset per partition. It measures how far behind a consumer group is from real-time.

```
# CLI monitoring
kafka-consumer-groups.sh --bootstrap-server localhost:9092 \
  --describe --group order-processing-group

# Output:
# TOPIC         PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG
# order-events  0          10450           10500           50
# order-events  1          10300           10300           0    <- healthy
# order-events  2          9800            10200           400  <- concerning!
```

Monitoring Stack

- **Burrow:** LinkedIn's lag evaluator. Analyzes lag TREND (not just absolute value). A stable lag of 100 is fine; lag growing by 100/min is an emergency.
- **Prometheus + JMX Exporter + Grafana:** Expose *kafka_consumer_group_lag* metric. Alert when lag exceeds threshold for sustained period.
- **Spring Boot Actuator:** Custom health indicator checking lag via *AdminClient.listConsumerGroupOffsets()*.

■■ **Trap:** Growing lag means the consumer is falling behind. If lag exceeds retention period, records are **permanently lost** (deleted before consumed). Monitor lag TREND, not just absolute value.

Q33. What is the difference between KTable and KStream in Kafka Streams?

Medium Kafka Streams

Answer:

Aspect	KStream	KTable
Represents	Unbounded stream of events. Every record is an independent fact (append-only log).	Changelog of updates per key. Latest value per key replaces previous (upsert semantics).
Analogy	INSERT: every event appended	UPSERT: latest value per key is the 'current state'
Backed by	Regular Kafka topic	Compacted topic (latest per key)
Null value	Valid record (data with null)	Delete (tombstone) — removes key from the table
Example	OrderCreated, OrderShipped, OrderCancelled events	Current order status per orderId; only latest status matters
Operations	map, filter, flatMap, groupByKey, join	Same + aggregate, reduce, toStream conversion

```
// KStream: process every event independently
KStream<String, OrderEvent> orders = builder.stream("order-events");
orders.filter((key, val) -> val.getAmount() > 1000)
      .to("high-value-orders");

// KTable: maintain current state per key
KTable<String, UserProfile> users = builder.table("user-profiles");
// users always has the LATEST profile for each userId

// Stream-Table Join: enrich events with current state
KStream<String, EnrichedOrder> enriched = orders.join(users,
    (order, user) -> new EnrichedOrder(order, user.getName()));
// Each order event is enriched with the user's CURRENT profile
```

■ **Follow-up:** 'When do you use GlobalKTable?' When you need to join with a small reference dataset that ALL stream instances need complete access to (like country codes, currency rates). Unlike KTable (partitioned), GlobalKTable is replicated to every instance.

Q34. How do you design a Kafka topic naming convention and partition strategy for microservices?

Medium System Design

Answer:

Topic Naming Convention

```
# Pattern: <domain>.<entity>.<event-type>
orders.order.created          # 12 partitions, RF=3, retention=7d
orders.order.shipped          # 12 partitions, RF=3
payments.payment.completed    # 12 partitions, RF=3
payments.payment.failed       # 6 partitions, RF=3
inventory.stock.reserved      # 12 partitions, RF=3
notifications.email.requested # 6 partitions, RF=3

# System topics (auto-created by Spring Kafka)
orders.order.created-retry-0  # Retry topic
orders.order.created-dlt      # Dead Letter Topic

# State/Changelog topics (compacted)
users.user.profile            # cleanup.policy=compact, infinite retention
```

Partition Strategy

- **Partition key = aggregate ID** (orderId, userId) for per-entity ordering.
- **Partition count = 2-3x peak consumer count.** You can increase later but NEVER decrease.
- **Replication factor = 3** for all production topics. *min.insync.replicas=2*.
- **Retention:** 7d for events, infinite + compaction for state topics.

Workload	Recommended Partitions	Reasoning
Low volume, few consumers	3-6	Simple, low broker overhead
Moderate production system	6-12	Good parallelism, manageable rebalances
High-throughput pipeline	30-50	Scales to many consumers across pods
Extreme scale (100K+ msg/s)	100+	Distributed across many brokers/disks

■■ **Trap:** Can you reduce partition count? **NO** — Kafka does not support it. Increasing partitions also re-keys data (same key may map to different partition). Plan carefully upfront. Over-provision by 2-3x.

■ This completes **Part 2 (Questions 18–34)** covering Avro, Schema Registry, Schema Evolution, Spring Boot Avro Integration, Producer Tuning, Outbox Pattern, Backpressure, Batch Listeners, Log Compaction, DLT Replay, Disk Storage, Consumer Lag, Kafka Streams (KTable/KStream), and Topic Design. **Part 3** will cover Questions 35–51: Security, Monitoring, Exactly-Once Deep Dive, Custom Partitioners, Interceptors, and more. Say **'next'** to continue!

100 Kafka Interview

Questions & Answers

with Detailed Code Examples

Part 3: Questions 35 – 51

Security • Monitoring • Headers • Custom Partitioner • Interceptors

Serializers • Static Membership • MirrorMaker • Kubernetes • Health Checks

by **TechTribers**

@techtribes • www.techtriber.com

Q35. How does Kafka Security work? Explain SSL/TLS, SASL, and ACLs.

Hard Security

Answer:

Kafka security has three layers: **encryption** (SSL/TLS), **authentication** (SASL), and **authorization** (ACLs). All three are independent and can be combined.

Layer	Mechanism	What It Does	Config
Encryption	SSL/TLS	Encrypts data in transit between clients and brokers, and between brokers (inter-broker)	listeners=SSL://... ssl.keystore.location ssl.truststore.location
Authentication	SASL/PLAIN SASL/SCRAM SASL/GSSAPI SASL/OAUTHBEARER	Verifies client identity. PLAIN=username/password. SCRAM=salted challenge. GSSAPI=Kerberos. OAUTH=token-based.	sasl.mechanism=SCRAM-SHA-256 security.protocol=SASL_SSL listener.name.sasl_ssl...
Authorization	ACLs (Access Control Lists)	Controls which authenticated principals can READ, WRITE, CREATE, DELETE on topics/groups	authorizer.class.name= ...AclAuthorizer super.users=User:admin

```
# Grant WRITE to producer, READ to consumer (kafka-acls.sh)
kafka-acls.sh --bootstrap-server broker:9092 --command-config admin.properties \
  --add --allow-principal User:order-producer \
  --operation Write --topic order-events

kafka-acls.sh --bootstrap-server broker:9092 --command-config admin.properties \
  --add --allow-principal User:order-consumer \
  --operation Read --topic order-events \
  --group order-processing-group

# Spring Boot client config for SASL_SSL
spring:
  kafka:
    properties:
      security.protocol: SASL_SSL
      sasl.mechanism: SCRAM-SHA-256
      sasl.jaas.config: >
        org.apache.kafka.common.security.scram.ScramLoginModule required
        username="order-service" password="${KAFKA_PASSWORD}";
```

■ **Pro Tip:** Production standard: **SASL_SSL** (authentication + encryption together). Use SCRAM-SHA-256 over PLAIN because SCRAM doesn't transmit the password in cleartext. Always combine with ACLs for least-privilege access.

Q36. What are Kafka record headers? How do you use them for tracing and metadata?

Medium Headers

Answer:

Kafka record headers are **key-value metadata pairs** attached to each record without modifying the payload. They're ideal for cross-cutting concerns like tracing, routing, and versioning.

Common Use Cases

- **Distributed tracing:** Propagate correlation IDs (traceId, spanId) for observability.
- **Schema version:** Include schemaVersion header for multi-version consumers.
- **Source metadata:** Track origin service, timestamp, environment.
- **Content type:** Indicate payload format (avro, json, protobuf).
- **Routing:** DLT headers (original topic, exception, retry count) added by Spring Kafka.

```
// Java Producer – adding custom headers
ProducerRecord<String, String> record =
    new ProducerRecord<>("order-events", orderId, payload);
record.headers()
    .add("X-Correlation-Id", correlationId.getBytes())
    .add("X-Source-Service", "order-service".getBytes())
    .add("X-Schema-Version", "2".getBytes());
producer.send(record);

// Spring Boot Consumer – reading headers
@KafkaListener(topics = "order-events")
public void consume(@Payload String message,
                    @Header("X-Correlation-Id") byte[] correlationId,
                    @Header(KafkaHeaders.RECEIVED_PARTITION) int partition,
                    Acknowledgment ack) {
    MDC.put("correlationId", new String(correlationId));
    log.info("Processing order with traceId={}", new String(correlationId));
    orderService.process(message);
    ack.acknowledge();
}
```

■ **Follow-up:** 'How do you propagate tracing across Kafka?' Use a ProducerInterceptor to inject traceId from MDC into headers on send, and a ConsumerInterceptor to extract and set MDC on consume. Spring Cloud Sleuth/Micrometer Tracing does this automatically.

Q37. How do you write a custom Partitioner in Kafka? When would you need one?

Medium Producer

Answer:

The default partitioner uses *murmur2(key) % numPartitions*. A custom partitioner is needed when you require **non-standard routing logic**: priority-based routing, geographic affinity, or weighted distribution.

```
// Custom Partitioner: route VIP orders to partition 0 for priority processing
public class PriorityPartitioner implements Partitioner {

    @Override
    public int partition(String topic, Object key, byte[] keyBytes,
                         Object value, byte[] valueBytes, Cluster cluster) {
        int numPartitions = cluster.partitionCountForTopic(topic);

        if (key == null) return ThreadLocalRandom.current().nextInt(numPartitions);

        String keyStr = (String) key;

        // VIP orders always go to partition 0 (dedicated consumer)
        if (keyStr.startsWith("VIP-")) return 0;

        // Geographic routing: US -> partitions 1-3, EU -> partitions 4-6
        if (keyStr.startsWith("US-"))
            return 1 + (Math.abs(keyStr.hashCode()) % 3);
        if (keyStr.startsWith("EU-"))
            return 4 + (Math.abs(keyStr.hashCode()) % 3);

        // Default: standard hash for remaining keys
        return Math.abs(Utils.murmur2(keyBytes)) % numPartitions;
    }

    @Override public void configure(Map<String, ?> configs) {}
    @Override public void close() {}
}

// Register in producer config:
props.put(ProducerConfig.PARTITIONER_CLASS_CONFIG,
    "com.techtribers.kafka.PriorityPartitioner");
```

■■ **Trap:** Custom partitioners break the standard key-partition guarantee. If you change partitioner logic, existing keys may route to different partitions — breaking ordering for in-flight data. Test thoroughly and consider the migration path.

Q38. What are Kafka Interceptors? Write a producer and consumer interceptor.

Medium Advanced

Answer:

Interceptors are **hooks** that execute before/after send (producer) and before/after consume (consumer). They're used for cross-cutting concerns without modifying business logic.

Interceptor	Methods	Use Cases
ProducerInterceptor	onSend() — before serialization onAcknowledgement() — after ack	Add tracing headers, inject timestamps, log send metrics, modify records before send
ConsumerInterceptor	onConsume() — after poll, before processing onCommit() — after offset commit	Extract tracing context, log consumption metrics, filter records, record lag stats

```
// Producer Interceptor: inject tracing + timing
public class TracingProducerInterceptor
        implements ProducerInterceptor<String, String> {

    @Override
    public ProducerRecord<String, String> onSend(ProducerRecord<String, String> record) {
        record.headers().add("X-Send-Timestamp",
            Long.toString(System.currentTimeMillis()).getBytes());
        record.headers().add("X-Trace-Id",
            MDC.get("traceId") != null ? MDC.get("traceId").getBytes() : "none".getBytes());
        return record;  // Modified record continues to serializer
    }

    @Override
    public void onAcknowledgement(RecordMetadata meta, Exception ex) {
        if (ex != null) Metrics.counter("kafka.send.error").increment();
        else Metrics.counter("kafka.send.success").increment();
    }
    // configure() and close() omitted for brevity
}

// Consumer Interceptor: extract tracing context
public class TracingConsumerInterceptor
        implements ConsumerInterceptor<String, String> {

    @Override
    public ConsumerRecords<String, String> onConsume(ConsumerRecords<String, String> records) {
        for (ConsumerRecord<String, String> r : records) {
            Header traceHeader = r.headers().lastHeader("X-Trace-Id");
            if (traceHeader != null) MDC.put("traceId", new String(traceHeader.value()));
        }
        return records;
    }
    @Override public void onCommit(Map<TopicPartition, OffsetAndMetadata> offsets) {}
}

// Register in properties:
props.put(ProducerConfig.INTERCEPTOR_CLASSES_CONFIG,
    "com.techtribers.kafka.TracingProducerInterceptor");
```

■ **Pro Tip:** Interceptors are orthogonal to business logic — no annotation changes needed. Use them for: tracing propagation, metrics collection, audit logging, header injection. They work at the client library level, not Spring level.

Q39. What is static group membership? How does it reduce rebalances in Kubernetes?

Medium Consumer

Answer:

Aspect	Dynamic Membership (default)	Static Membership
Config	No group.instance.id set	group.instance.id=${HOSTNAME}
On restart/ rolling deploy	Consumer leaves group → rebalance → all partitions redistributed	Consumer rejoins with same instance ID → NO rebalance → same partitions reassigned
Rebalance count (3-pod deploy)	3 full rebalances (one per pod restart)	0 rebalances (if restart completes within session timeout)
Zombie detection	session.timeout.ms (heartbeat-based)	session.timeout.ms must be set higher than pod restart time (~60-120s)
Best for	Dev/test environments	Production Kubernetes, rolling deployments

```
# Spring Boot application.yml for Kubernetes static membership
spring:
  kafka:
    consumer:
      properties:
        group.instance.id: ${HOSTNAME}        # pod-name-0, pod-name-1, etc.
        session.timeout.ms: 120000            # 2 min > pod restart time
        partition.assignment.strategy: >
          org.apache.kafka.clients.consumer.CooperativeStickyAssignor

# During rolling deploy:
# 1. Pod-0 restarts -> broker waits 120s before removing from group
# 2. Pod-0 comes back with same group.instance.id -> same partitions
# 3. NO rebalance triggered -> other pods continue processing
# Result: zero-downtime deployments with no consumer disruption
```

■ **Follow-up:** 'How do you set group.instance.id uniquely per pod?' Use the pod hostname: *${HOSTNAME}* in K8s StatefulSets (pod-0, pod-1, pod-2). For Deployments, use *${POD_NAME}* from the Downward API. Each consumer must have a unique instance ID within the group.

Q40. How do you write a custom Serializer and Deserializer in Kafka?

Medium Serialization

Answer:

Custom serializers convert Java objects to bytes (producer-side) and back (consumer-side). Useful when you need specific encoding, compression, or format not provided by built-in serializers.

```
// Custom JSON Serializer using Jackson
public class OrderSerializer implements Serializer<Order> {
    private final ObjectMapper mapper = new ObjectMapper();

    @Override
    public byte[] serialize(String topic, Order order) {
        if (order == null) return null;
        try {
            return mapper.writeValueAsBytes(order);
        } catch (JsonProcessingException e) {
            throw new SerializationException("Failed to serialize Order", e);
        }
    }
}

// Custom JSON Deserializer
public class OrderDeserializer implements Deserializer<Order> {
    private final ObjectMapper mapper = new ObjectMapper();

    @Override
    public Order deserialize(String topic, byte[] data) {
        if (data == null) return null;
        try {
            return mapper.readValue(data, Order.class);
        } catch (IOException e) {
            throw new SerializationException("Failed to deserialize Order", e);
        }
    }
}

// Register in producer/consumer config:
props.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG,
    "com.techtribers.kafka.OrderSerializer");
props.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG,
    "com.techtribers.kafka.OrderDeserializer");

// Spring Boot: Use JsonSerializer/JsonDeserializer (built-in)
spring:
  kafka:
    producer:
      value-serializer: org.springframework.kafka.support.serializer.JsonSerializer
    consumer:
      value-deserializer: org.springframework.kafka.support.serializer.JsonDeserializer
      properties:
        spring.json.trusted.packages: com.techtribers.model
```

■■ **Trap:** Always handle null in serialize/deserialize — Kafka sends nulls for tombstone records (log compaction). Also: never throw unchecked exceptions from deserialize() without wrapping in SerializationException, or you'll crash the consumer.

Q41. What is the ErrorHandlingDeserializer and why is it critical for production?

Hard Error Handling

Answer:

The **ErrorHandlingDeserializer** is a Spring Kafka wrapper that catches deserialization exceptions and converts them into **handleable errors** instead of crashing the consumer thread.

Without ErrorHandlingDeserializer

- A single malformed record throws an exception inside *KafkaConsumer.poll()*.
- The exception occurs BEFORE your @KafkaListener method is called.
- Your error handler never sees it. The consumer thread crashes or enters an infinite retry loop.

With ErrorHandlingDeserializer

- Wraps the real deserializer. Catches any exception during deserialization.
- Stores the exception in a header (*__TypeId__deserialization_error*).
- Passes a null payload to your listener. The DefaultErrorHandler can then route it to DLT.

```
# Spring Boot application.yml config
spring:
  kafka:
    consumer:
      key-deserializer: org.springframework.kafka.support.serializer.ErrorHandlingDeserializer
      value-deserializer: org.springframework.kafka.support.serializer.ErrorHandlingDeserializer
      properties:
        spring.deserializer.key.delegate.class: org.apache.kafka.common.serialization.StringDeserializer
        spring.deserializer.value.delegate.class: io.confluent.kafka.serializers.KafkaAvroDeserializer

# Now: corrupted bytes -> caught by ErrorHandlingDeserializer
#   -> DeserializationException stored in header
#   -> DefaultErrorHandler sees it -> routes to DLT
#   -> Consumer continues processing next record!
```

■ **Pro Tip:** This is a **must-have** for any production consumer. Without it, a single bad record can take down your entire consumer. It's the 'seatbelt' of Kafka consumers — you don't need it until you REALLY need it.

Q42. What are the key Kafka JMX metrics to monitor in production?

Medium Monitoring

Answer:

Metric	Component	What It Measures	Alert Threshold
UnderReplicatedPartitions	Broker	Partitions where followers are behind the leader	Any value > 0
ActiveControllerCount	Broker	Number of active controllers. Should be exactly 1.	!= 1 (split brain or no controller)
RequestHandlerAvgIdlePercent	Broker	Broker request thread utilization (100% = idle)	< 30% (overloaded)
BytesInPerSec / BytesOutPerSec	Broker	Network throughput in/out. Baseline and watch spikes.	Deviation > 50% from baseline
records-lag-max	Consumer	Max consumer lag across all assigned partitions	Growing trend or exceeding threshold
records-consumed-rate	Consumer	Records consumed per second. Drop = stalled consumer.	Sudden drop to 0
record-send-rate	Producer	Records produced per second.	Drop below baseline
record-error-rate	Producer	Failed sends per second.	Any value > 0

```
# Prometheus JMX Exporter config for Kafka broker
rules:
  - pattern: 'kafka.server<type=ReplicaManager,name=UnderReplicatedPartitions>'
    name: kafka_server_underreplicated_partitions
  - pattern: 'kafka.controller<type=KafkaController,name=ActiveControllerCount>'
    name: kafka_controller_active_count
  - pattern: 'kafka.server<type=BrokerTopicMetrics,name=(.+)PerSec>'
    name: kafka_server_broker_topic_metrics_$1_per_sec

# Spring Boot Actuator — expose consumer metrics
management:
  endpoints.web.exposure.include: health,prometheus
  metrics.tags.application: order-service
```

■ **Follow-up:** 'What is the single most important metric?' **UnderReplicatedPartitions**. If > 0, it means data is at risk — ISR shrinkage means the cluster can't sustain broker failures. It's the canary in the coal mine for disk, network, or broker issues.

Q43. How do you implement a Kafka health check in Spring Boot?

Medium Spring Boot

Answer:

```
// Custom Kafka Health Indicator — checks broker connectivity + consumer lag
@Component
public class KafkaHealthIndicator implements HealthIndicator {

    private final AdminClient adminClient;
    private final KafkaListenerEndpointRegistry registry;

    public KafkaHealthIndicator(KafkaAdmin kafkaAdmin,
                                KafkaListenerEndpointRegistry registry) {
        this.adminClient = AdminClient.create(kafkaAdmin.getConfigurationProperties());
        this.registry = registry;
    }

    @Override
    public Health health() {
        try {
            // 1. Check broker connectivity
            DescribeClusterResult cluster = adminClient.describeCluster();
            int brokerCount = cluster.nodes().get(5, TimeUnit.SECONDS).size();

            // 2. Check all listener containers are running
            boolean allRunning = registry.getListenerContainers().stream()
                .allMatch(c -> c.isRunning());

            // 3. Check consumer lag via AdminClient
            Map<String, Object> details = new HashMap<>();
            details.put("brokers", brokerCount);
            details.put("listenersRunning", allRunning);
            details.put("clusterId", cluster.clusterId().get(5, SECONDS));

            if (brokerCount == 0 || !allRunning) {
                return Health.down().withDetails(details).build();
            }
            return Health.up().withDetails(details).build();

        } catch (Exception e) {
            return Health.down().withException(e).build();
        }
    }
}

// Kubernetes liveness/readiness probes:
// GET /actuator/health/liveness   -> always UP if JVM running
// GET /actuator/health/readiness  -> includes Kafka health check
```

■ **Pro Tip:** For Kubernetes: use the health indicator in the **readiness probe** (not liveness). If Kafka is temporarily down, you want the pod to stop receiving traffic (readiness=DOWN) but NOT restart (liveness stays UP). Restarting won't fix a Kafka outage.

Q44. What is Kafka MirrorMaker 2? How do you replicate data across clusters?

Medium Operations

Answer:

MirrorMaker 2 (MM2) is Kafka's built-in tool for replicating data between clusters. It's built on the Kafka Connect framework and supports active-active, active-passive, and hub-spoke topologies.

Feature	MirrorMaker 1 (legacy)	MirrorMaker 2
Architecture	Embedded consumer + producer	Kafka Connect-based connectors
Offset sync	No offset translation	Automatic offset translation (consumer can failover)
Topic creation	Manual	Automatic (mirrors config)
Topology	Active-passive only	Active-active, fan-out, hub-spoke, aggregation
Topic naming	Same name (collisions)	Prefixed: source.topic-name (prevents loops)

```
# mm2.properties for active-passive replication
clusters = primary, dr
primary.bootstrap.servers = primary-kafka:9092
dr.bootstrap.servers = dr-kafka:9092

# Replicate all topics from primary -> DR
primary->dr.enabled = true
primary->dr.topics = .*            # All topics (regex)
primary->dr.groups = .*            # All consumer groups

# Topic naming: primary.order-events -> dr cluster
replication.policy.class = org.apache.kafka.connect.mirror.DefaultReplicationPolicy

# Sync consumer offsets every 10 seconds
sync.group.offsets.enabled = true
sync.group.offsets.interval.seconds = 10

# Heartbeat and checkpoint topics for monitoring
emit.heartbeats.enabled = true
emit.checkpoints.enabled = true
```

■ **Follow-up:** 'How do consumers failover to the DR cluster?' MM2 syncs consumer group offsets with automatic translation (source offset → DR offset). After failover, consumers resume from the translated offset with minimal data loss. Check checkpoint topics for RPO monitoring.

Q45. How do you run Kafka in Kubernetes? What are the key operational considerations?

Medium Kubernetes

Answer:

Key Design Decisions

- **StatefulSet** (not Deployment): Kafka needs stable network IDs (broker-0, broker-1) and persistent storage. StatefulSet provides both.
- **PersistentVolumeClaims:** Each broker needs its own PV for log data. Use SSDs (*storageClassName: ssd*) for performance.
- **Headless Service:** Each broker needs a stable DNS name for inter-broker communication: *broker-0.kafka-headless.namespace.svc.cluster.local*.
- **Anti-affinity:** Spread brokers across nodes/AZs to survive node failures.
- **Resource requests:** Set CPU/memory requests = limits to avoid OOM kills and noisy neighbors.

```
# Strimzi Kafka operator — declarative Kafka cluster in K8s
apiVersion: kafka.strimzi.io/v1beta2
kind: Kafka
metadata:
  name: production-cluster
spec:
  kafka:
    replicas: 3
    version: 3.7.0
    listeners:
      - name: plain
        port: 9092
        type: internal
        tls: false
    storage:
      type: persistent-claim
      size: 500Gi
      class: ssd
    config:
      min.insync.replicas: 2
      default.replication.factor: 3
      num.partitions: 12
    template:
      pod:
        affinity:
          podAntiAffinity:
            requiredDuringSchedulingIgnoredDuringExecution:
              - labelSelector:
                  matchLabels:
                    strimzi.io/name: production-cluster-kafka
                topologyKey: topology.kubernetes.io/zone
```

■ **Pro Tip:** Use the **Strimzi operator** — it handles broker configuration, rolling upgrades, TLS certificate rotation, topic management, and user provisioning declaratively. Don't deploy Kafka manually in K8s; the operational complexity is massive.

Q46. What is the __consumer_offsets topic? How does Kafka store consumer group state?

Medium Internals

Answer:

The **__consumer_offsets** topic is an internal, compacted Kafka topic that stores committed consumer group offsets. When a consumer calls *commitSync()*, the offset is written here.

- **Partitions:** 50 by default (*offsets.topic.num.partitions=50*).
- **Replication:** RF=3 (*offsets.topic.replication.factor=3*).
- **Cleanup:** *compact* — only the latest offset per (group, topic, partition) is retained.
- **Group coordinator:** Each consumer group is assigned to the broker hosting the partition: *hash(groupId) % 50*.

```
// What's stored in __consumer_offsets:
// Key:   (group_id, topic, partition)
// Value: (offset, metadata, commit_timestamp, leader_epoch)

// Example entries:
// Key: ("order-group", "order-events", 0) -> Value: (offset=10500)
// Key: ("order-group", "order-events", 1) -> Value: (offset=10300)

// The group coordinator for "order-group" is the broker
// hosting partition: hash("order-group") % 50 = e.g., partition 17
// That broker manages group membership, rebalancing, and offsets.

# View consumer group state:
kafka-consumer-groups.sh --bootstrap-server broker:9092 \
  --describe --group order-group --state
```

■ **Follow-up:** 'What happens if __consumer_offsets becomes unavailable?' Consumer group coordination fails — no commits, no rebalances. This is why it has RF=3. Monitor its under-replicated partitions. If lag builds on __consumer_offsets, the entire cluster's consumer groups are degraded.

Q47. What is auto.create.topics.enable and why should you disable it in production?

Easy Operations

Answer:

When *auto.create.topics.enable=true* (default), Kafka automatically creates a topic the first time a producer sends to it or a consumer subscribes to it. This is **dangerous in production**.

Why Disable

- **Typos create topics:** Producer sends to 'order-evnts' instead of 'order-events' → a garbage topic is silently created. No error returned.
- **Wrong defaults:** Auto-created topics use broker defaults (partitions=1, RF=1). Production topics need explicit partition count, RF=3, and retention settings.
- **No governance:** Any client can create any topic. No naming convention enforcement, no review process.

```
# Broker config: DISABLE auto-creation
auto.create.topics.enable=false

# Instead: create topics explicitly with correct settings
kafka-topics.sh --create \
  --bootstrap-server broker:9092 \
  --topic order-events \
  --partitions 12 \
  --replication-factor 3 \
  --config min.insync.replicas=2 \
  --config retention.ms=604800000    # 7 days

# Spring Boot: auto-create via KafkaAdmin bean
@Bean
public NewTopic orderEventsTopic() {
    return TopicBuilder.name("order-events")
        .partitions(12).replicas(3)
        .config(TopicConfig.MIN_IN_SYNC_REPLICAS_CONFIG, "2")
        .build();
}
```

■■ **Trap:** In production, ALWAYS set *auto.create.topics.enable=false*. Manage topics via IaC (Terraform, Strimzi TopicOperator) or CI/CD pipelines. Every topic should be version-controlled with explicit configuration.

Q48. What are Kafka quotas? How do you prevent a single client from overwhelming the cluster?

Medium Operations

Answer:

Kafka quotas are **rate limits** applied to clients to prevent a single producer or consumer from monopolizing broker resources.

Quota Type	What It Limits	Config	Effect When Exceeded
Produce rate	Bytes/sec a producer can send per broker	producer_byte_rate (e.g., 10MB/s)	Broker throttles: delays response, producer backs off
Fetch rate	Bytes/sec a consumer can fetch per broker	consumer_byte_rate (e.g., 20MB/s)	Broker delays fetch responses to slow consumer
Request rate	% of broker request handler threads	request_percentage (e.g., 25%)	Limits CPU time allocated to the client

```
# Set quotas per client-id (kafka-configs.sh)
kafka-configs.sh --bootstrap-server broker:9092 \
  --alter --add-config \
  'producer_byte_rate=10485760,consumer_byte_rate=20971520' \
  --entity-type clients --entity-name order-producer

# Set default quotas for all clients
kafka-configs.sh --bootstrap-server broker:9092 \
  --alter --add-config \
  'producer_byte_rate=5242880' \
  --entity-type clients --entity-default

# Monitor throttle metrics:
# kafka.server:type=Request,name=ThrottleTimeMs
# kafka.server:type=Produce,name=ThrottleTimeMs
```

■ **Follow-up:** 'When do quotas matter?' Multi-tenant clusters, shared infrastructure, or when a data pipeline backfill could saturate network bandwidth. Without quotas, a single bulk import can starve all other producers/consumers.

Q49. How does the Spring Kafka concurrency model work? What is the relationship between concurrency and partitions?

Hard Spring Boot

Answer:

When you set *concurrency=N* in Spring Kafka, the *ConcurrentMessageListenerContainer* creates **N child containers**, each with its own *KafkaConsumer* instance and dedicated thread.

Concurrency	Partitions	Result
3	12	Each consumer thread gets 4 partitions. Full utilization.
3	3	Each consumer thread gets 1 partition. Optimal 1:1 mapping.
6	3	3 threads get 1 partition each. 3 threads sit IDLE (wasted resources).
1	12	Single thread processes all 12 partitions sequentially. Bottleneck.
12	12	1:1 mapping. Maximum parallelism. But 12 threads + 12 connections.

```
// Global concurrency in application.yml
spring.kafka.listener.concurrency: 6

// Per-listener concurrency override
@KafkaListener(topics = "high-volume-topic",
               concurrency = "12",     // Override global
               groupId = "hv-group")
public void handle(String msg, Acknowledgment ack) {
    process(msg);
    ack.acknowledge();
}

// Multi-pod deployment: 3 pods x concurrency=4 = 12 total consumers
// With 12 partitions: each pod gets 4 partitions (4 threads each)
// Rule: total consumers across all pods <= partition count
```

■■ **Trap:** concurrency > partition count = wasted threads. Each excess thread creates a KafkaConsumer that sits idle, consuming memory and connections. Always: total consumers (pods x concurrency) <= partition count.

Q50. What is the difference between auto.offset.reset=earliest and latest? When do they apply?

Easy Consumer

Answer:

auto.offset.reset controls what happens when a consumer starts and **no committed offset exists** for its group on a partition. It does NOT apply on every restart.

Value	Behavior	When It Applies	Use Case
earliest	Start from the very first available record in the partition (offset 0)	Only when NO committed offset exists for this group+partition (first time, or offsets expired)	Data pipelines, reprocessing, new consumer groups that need historical data
latest	Start from the end of the partition (only new records after consumer starts)	Same as above: only when no committed offset exists	Real-time alerting, live dashboards where historical data is irrelevant

none	Throw an exception if no committed offset exists	Same trigger condition	Fail-fast: forces explicit offset management

```
// Common confusion: "I set earliest but it's not re-reading!"
// auto.offset.reset ONLY triggers when there's NO committed offset.
// If the consumer group already has a committed offset, Kafka resumes
// from that offset regardless of this setting.

// To truly re-read from beginning:
kafka-consumer-groups.sh --bootstrap-server broker:9092 \
  --group order-group --reset-offsets --to-earliest \
  --topic order-events --execute

// Or programmatically:
consumer.seekToBeginning(consumer.assignment());
```

■■ **Trap:** Biggest trap: candidates think 'earliest' means 're-read everything on every restart.' NO — it only applies when **no committed offset exists**. Once a group has committed offsets, Kafka resumes from the last commit. To replay, you must explicitly reset offsets.

Q51. How do you test Kafka applications? Compare EmbeddedKafka, Testcontainers, and MockSchemaRegistry.

Hard Testing

Answer:

Approach	What It Is	Speed	Fidelity	Best For
EmbeddedKafka (@EmbeddedKafka)	In-process Kafka broker running in test JVM. Spring Boot integration.	Fast (~3-5s startup)	Medium — not real Kafka; some behavior differs	Unit/integration tests. CI pipelines where Docker isn't available.
Testcontainers	Real Kafka (+ SR) in Docker containers. Full production parity.	Slow (~10-20s startup)	High — real Kafka broker and Schema Registry	Integration tests. Schema evolution tests. Pre-prod validation.
Mock Schema Registry	In-memory SR using mock:// URL scheme. No HTTP server needed.	Instant	Low — only tests serde, not compat rules	Unit tests for Avro serialization/deser. Fast feedback loop.

```
// 1. EmbeddedKafka Spring Boot test
@SpringBootTest
@EmbeddedKafka(partitions = 3, topics = {"order-events"})
class OrderConsumerTest {
    @Autowired KafkaTemplate<String, String> template;
    @Autowired OrderConsumer consumer;

    @Test
    void shouldProcessOrder() {
        template.send("order-events", "key1", "{\"orderId\":\"1\"}");
        await().atMost(10, SECONDS)
            .untilAsserted(() -> verify(orderService).process(any()));
    }
}

// 2. Testcontainers — real Kafka + Schema Registry
@Testcontainers
@SpringBootTest
class AvroIntegrationTest {
    @Container
    static KafkaContainer kafka = new KafkaContainer(
        DockerImageName.parse("confluentinc/cp-kafka:7.6.0"));
    @Container
    static GenericContainer<?> schemaRegistry = new GenericContainer<>(
        "confluentinc/cp-schema-registry:7.6.0")
        .withExposedPorts(8081)
        .withEnv("SCHEMA_REGISTRY_KAFKASTORE_BOOTSTRAP_SERVERS", kafka...);

    @DynamicPropertySource
    static void props(DynamicPropertyRegistry reg) {
        reg.add("spring.kafka.bootstrap-servers", kafka::getBootstrapServers);
    }
}

// 3. Mock Schema Registry for unit tests
props.put("schema.registry.url", "mock://test-registry");
// Uses in-memory registry — no HTTP, instant, great for unit tests
```

■ **Pro Tip:** Layered testing strategy: (1) Mock SR for fast unit tests of serialization. (2) EmbeddedKafka for Spring integration tests in CI. (3) Testcontainers for full end-to-end with real SR and compatibility checks. Use all three layers.

■ This completes **Part 3 (Questions 35–51)** covering Security (SSL/SASL/ACLs), Headers & Tracing, Custom Partitioners, Interceptors, Static Membership, Custom Serializers, ErrorHandlingDeserializer, JMX Monitoring, Health Checks, MirrorMaker 2, Kubernetes Deployment, __consumer_offsets, Auto-Topic Creation, Quotas, Concurrency Model, auto.offset.reset, and Testing. **Part 4** will cover Questions 52–68. Say **'next'** to continue!

100 Kafka Interview

Questions & Answers

with Detailed Code Examples

Part 4: Questions 52 – 68

Windowing • Exactly-Once Streams • CQRS • Saga Pattern • CDC

Multi-Cluster • Tiered Storage • Rack Awareness • Consumer Seek • AdminClient

by **TechTribers**

@techtribes • www.techtriber.com

Q52. What are windowed operations in Kafka Streams? Explain Tumbling, Hopping, Sliding, and Session windows.

Hard Kafka Streams

Answer:

Window Type	Behavior	Size / Gap	Overlap	Example Use Case
Tumbling	Fixed, non-overlapping. Each record belongs to exactly one window.	Fixed size (e.g., 5 min)	None	Count orders per 5-min interval. Each order in exactly one bucket.
Hopping	Fixed size, advances by hop interval. Records can belong to multiple windows.	Size: 10 min Hop: 5 min	Yes	Rolling 10-min average, updated every 5 min. Smooth aggregations.
Sliding	Window around each record. Triggered by record arrival, not wall clock.	Size: 10 min Grace: 1 min	Yes	Exactly 2 events within 10 min of each other. Event correlation.
Session	Dynamic, inactivity-based. Closes after gap of no activity per key.	Inactivity gap (e.g., 30 min)	Merges	User session tracking. Merges events until 30 min of silence.

```
// Tumbling: fixed 5-minute windows, no overlap
KTable<Windowed<String>, Long> tumblingCounts = orders
    .groupByKey()
    .windowedBy(TimeWindows.ofSizeWithNoGrace(Duration.ofMinutes(5)))
    .count();

// Hopping: 10-minute window, advancing every 2 minutes
KTable<Windowed<String>, Long> hoppingCounts = orders
    .groupByKey()
    .windowedBy(TimeWindows.ofSizeAndGrace(Duration.ofMinutes(10), Duration.ofMinutes(1))
        .advanceBy(Duration.ofMinutes(2)))
    .count();

// Session: merge events within 30-min inactivity gap
KTable<Windowed<String>, Long> sessions = clicks
    .groupByKey()
    .windowedBy(SessionWindows.ofInactivityGapWithNoGrace(Duration.ofMinutes(30)))
    .count();  // Counts clicks per user session
```

■ **Follow-up:** 'What is grace period?' Time allowed for late-arriving records after window closes. *ofSizeWithNoGrace()* means strictly on time. With grace, late records still update the window. After grace expires, records are dropped. Critical for event-time processing with out-of-order data.

Q53. How does Exactly-Once Semantics (EOS) work in Kafka Streams?

Hard Kafka Streams

Answer:

Kafka Streams achieves EOS by wrapping the entire **read-process-write** cycle in a Kafka transaction. Each processing step atomically reads input, updates state store, and writes output.

How It Works Internally

- **1.** StreamThread begins a Kafka transaction.
- **2.** Reads records from input partitions (consumer offsets tracked).
- **3.** Processes records, updates RocksDB state stores, writes to output topics.
- **4.** Commits transaction atomically: output records + consumer offsets + state store changelog entries.
- **5.** If any step fails, entire transaction aborts. State rolls back via changelog replay.

```
// Enable EOS in Kafka Streams (one line!)
Properties props = new Properties();
props.put(StreamsConfig.PROCESSING_GUARANTEE_CONFIG,
    StreamsConfig.EXACTLY_ONCE_V2);  // Use v2 (Kafka 2.5+)

// Under the hood, this sets:
// - Transactional producer (transactional.id per task)
// - Consumer isolation.level = read_committed
// - Idempotent producer enabled
// - State store changelogs are transactional

// Spring Boot:
spring:
  kafka:
    streams:
      properties:
        processing.guarantee: exactly_once_v2
```

■■ **Trap:** EOS adds ~15-20% latency overhead due to transaction coordination. Use it when correctness is critical (financial aggregations, billing). For most stream processing, at-least-once with idempotent sinks is simpler and faster.

Q54. What is the Saga Pattern? How do you implement choreography-based sagas with Kafka?

Hard Design Patterns

Answer:

A **Saga** is a sequence of local transactions across microservices, where each step publishes an event that triggers the next. If any step fails, **compensating transactions** undo previous steps.

E-Commerce Saga: Order → Payment → Inventory → Shipping

```
// Happy path:
// OrderService: CreateOrder -> publishes "order.created"
// PaymentService: listens "order.created" -> charges card -> publishes "payment.completed"
// InventoryService: listens "payment.completed" -> reserves stock -> publishes "stock.reserved"
// ShippingService: listens "stock.reserved" -> creates shipment -> publishes "order.shipped"

// Failure path (payment fails):
// PaymentService: charge fails -> publishes "payment.failed"
// OrderService: listens "payment.failed" -> marks order CANCELLED (compensating txn)

// Failure path (stock unavailable):
// InventoryService: no stock -> publishes "stock.reservation.failed"
// PaymentService: listens "stock.reservation.failed" -> refunds payment (compensating)
// OrderService: listens "stock.reservation.failed" -> marks order CANCELLED
```

Spring Boot Implementation

```
// OrderService — starts the saga
@Service @Transactional
public class OrderSagaStarter {
    public void createOrder(CreateOrderRequest req) {
        Order order = orderRepo.save(Order.builder()
            .status(OrderStatus.PENDING).build());
        kafkaTemplate.send("order.created", order.getId().toString(),
            new OrderCreatedEvent(order.getId(), req.getUserId(), req.getAmount()));
    }

    @KafkaListener(topics = "payment.failed")
    public void onPaymentFailed(PaymentFailedEvent event) {
        orderRepo.updateStatus(event.getOrderId(), OrderStatus.CANCELLED);
        log.warn("Saga compensation: order {} cancelled", event.getOrderId());
    }
}

// PaymentService — saga step 2
@KafkaListener(topics = "order.created")
public void onOrderCreated(OrderCreatedEvent event) {
    try {
        paymentGateway.charge(event.getUserId(), event.getAmount());
        kafkaTemplate.send("payment.completed",
            new PaymentCompletedEvent(event.getOrderId(), event.getAmount()));
    } catch (PaymentException e) {
        kafkaTemplate.send("payment.failed",
            new PaymentFailedEvent(event.getOrderId(), e.getMessage()));
    }
}
```

■ **Follow-up:** 'Choreography vs Orchestration sagas?' Choreography: each service listens and reacts (decoupled, but complex flow tracking). Orchestration: a central coordinator (saga orchestrator) directs steps via commands (easier to reason about, single point of failure). Most teams start with choreography and move to orchestration as complexity grows.

Q55. What is CQRS? How do you build a CQRS read model using Kafka?

Hard Design Patterns

Answer:

CQRS (Command Query Responsibility Segregation) separates the write model (commands) from the read model (queries). Kafka acts as the event backbone connecting them.

- **Write side:** Handles commands (CreateOrder, UpdateOrder). Writes to the primary database. Publishes domain events to Kafka.
- **Kafka:** Carries events (OrderCreated, OrderUpdated) as the source of truth for state changes.
- **Read side:** Consumes events from Kafka. Builds denormalized, query-optimized read models (Elasticsearch, Redis, materialized views).

```
// Write side: publishes events via Outbox Pattern
@Transactional
public Order createOrder(CreateOrderRequest req) {
    Order order = orderRepo.save(Order.from(req));
    outboxRepo.save(new OutboxEvent("OrderCreated", order.toEvent()));
    return order;  // Debezium CDC -> Kafka
}

// Read side: builds denormalized view from events
@Service
public class OrderSearchProjection {

    @KafkaListener(topics = {"order.created", "order.shipped", "order.cancelled"})
    public void project(@Payload OrderEvent event,
                        @Header(KafkaHeaders.RECEIVED_TOPIC) String topic) {
        switch (topic) {
            case "order.created" -> elasticRepo.save(OrderDocument.builder()
                .orderId(event.getOrderId())
                .userId(event.getUserId())
                .status("CREATED")
                .amount(event.getAmount())
                .searchableText(event.getItemNames())
                .build());

            case "order.shipped" -> elasticRepo.updateStatus(
                event.getOrderId(), "SHIPPED", event.getTrackingNumber());

            case "order.cancelled" -> elasticRepo.delete(event.getOrderId());
        }
    }
}
// Query: GET /api/orders/search?q=laptop&status=SHIPPED
// -> hits Elasticsearch directly (fast, denormalized, pre-joined)
```

■ **Pro Tip:** CQRS with Kafka means the read model is **eventually consistent**. If the user creates an order and immediately searches, they might not find it (event hasn't been projected yet). Handle with: UI optimistic updates, or synchronous read-after-write for critical paths.

Q56. What is Change Data Capture (CDC)? How does Debezium work with Kafka?

Medium CDC

Answer:

CDC captures row-level changes (INSERT, UPDATE, DELETE) from a database's transaction log (WAL/binlog) and streams them to Kafka as events. **Debezium** is the most popular open-source CDC tool.

How Debezium Works

- **1.** Reads the database's write-ahead log (PostgreSQL WAL, MySQL binlog, MongoDB oplog).
- **2.** Converts each change into a structured event (before/after snapshots, operation type, timestamp).
- **3.** Publishes to Kafka topic named *{prefix}.{schema}.{table}*.
- **4.** Tracks position in the WAL via Kafka Connect offsets — exactly-once delivery on restart.

```
// Debezium PostgreSQL connector config (Kafka Connect JSON)
{
  "name": "pg-cdc-connector",
  "config": {
    "connector.class": "io.debezium.connector.postgresql.PostgresConnector",
    "database.hostname": "postgres",
    "database.port": "5432",
    "database.user": "debezium",
    "database.dbname": "orders_db",
    "database.server.name": "orderservice",
    "table.include.list": "public.orders,public.order_items",
    "plugin.name": "pgoutput",
    "slot.name": "debezium_slot",
    "publication.name": "dbz_publication",
    "topic.prefix": "cdc",
    "transforms": "unwrap",
    "transforms.unwrap.type": "io.debezium.transforms.ExtractNewRecordState"
  }
}
// Result: changes to public.orders -> topic "cdc.public.orders"
// Each event contains: { before: {...}, after: {...}, op: "u", ts_ms: ... }
```

■ **Follow-up:** 'CDC vs dual-write vs polling?' CDC reads from the WAL — zero impact on application code, captures ALL changes (including direct SQL), and is near-real-time. Polling misses changes between intervals. Dual-write risks inconsistency. CDC is the gold standard for database-to-Kafka integration.

Q57. How do you use AdminClient to manage Kafka topics and consumer groups programmatically?

Medium Java API

Answer:

```
import org.apache.kafka.clients.admin.*;
import java.util.*;

Properties props = new Properties();
props.put(AdminClientConfig.BOOTSTRAP_SERVERS_CONFIG, "localhost:9092");

try (AdminClient admin = AdminClient.create(props)) {

    // 1. Create a topic
    NewTopic topic = new NewTopic("order-events", 12, (short) 3)
        .configs(Map.of(
            "min.insync.replicas", "2",
            "retention.ms", "604800000",      // 7 days
            "cleanup.policy", "delete"));
    admin.createTopics(List.of(topic)).all().get();

    // 2. List all topics
    Set<String> topics = admin.listTopics().names().get();
    System.out.println("Topics: " + topics);

    // 3. Describe topic (partitions, replicas, ISR)
    Map<String, TopicDescription> desc =
        admin.describeTopics(List.of("order-events")).allTopicNames().get();
    desc.get("order-events").partitions().forEach(p ->
        System.out.printf("P%d leader=%d ISR=%s%n",
            p.partition(), p.leader().id(), p.isr()));

    // 4. Check consumer group lag
    Map<TopicPartition, OffsetAndMetadata> offsets =
        admin.listConsumerGroupOffsets("order-group")
             .partitionsToOffsetAndMetadata().get();
    Map<TopicPartition, ListOffsetsResult.ListOffsetsResultInfo> endOffsets =
        admin.listOffsets(offsets.keySet().stream().collect(
            Collectors.toMap(tp -> tp, tp -> OffsetSpec.latest()))).all().get();

    offsets.forEach((tp, oam) -> {
        long lag = endOffsets.get(tp).offset() - oam.offset();
        System.out.printf("%s P%d lag=%d%n", tp.topic(), tp.partition(), lag);
    });

    // 5. Delete consumer group (must be empty/inactive)
    admin.deleteConsumerGroups(List.of("old-group")).all().get();
}
```

■ **Pro Tip:** AdminClient is your Swiss Army knife for Kafka operations in Java. Use it for: health checks, dynamic topic creation in Spring Boot (@Bean NewTopic), consumer lag monitoring, and automated cleanup scripts.

Q58. How do you seek to a specific offset or timestamp in a Kafka consumer?

Medium Consumer

Answer:

```
// 1. Seek to specific offset
consumer.assign(List.of(new TopicPartition("order-events", 0)));
consumer.seek(new TopicPartition("order-events", 0), 5000L); // Start at offset 5000

// 2. Seek to beginning (re-read all data)
consumer.seekToBeginning(consumer.assignment());

// 3. Seek to end (skip to latest, only new records)
consumer.seekToEnd(consumer.assignment());

// 4. Seek by timestamp (e.g., re-read from yesterday)
Map<TopicPartition, Long> timestamps = new HashMap<>();
long yesterday = Instant.now().minus(1, ChronoUnit.DAYS).toEpochMilli();
consumer.assignment().forEach(tp -> timestamps.put(tp, yesterday));

Map<TopicPartition, OffsetAndTimestamp> offsets =
    consumer.offsetsForTimes(timestamps);  // Finds offset >= timestamp

offsets.forEach((tp, ot) -> {
    if (ot != null) consumer.seek(tp, ot.offset());
});

// Spring Kafka: ConsumerSeekAware for programmatic seek
@Service
public class ReplayConsumer implements ConsumerSeekAware {
    private ConsumerSeekCallback seekCallback;

    @Override
    public void registerSeekCallback(ConsumerSeekCallback callback) {
        this.seekCallback = callback;
    }

    public void replayFromYesterday(String topic, int partition) {
        long yesterday = Instant.now().minus(1, ChronoUnit.DAYS).toEpochMilli();
        seekCallback.seekToTimestamp(topic, partition, yesterday);
    }

    @KafkaListener(topics = "order-events")
    public void consume(String msg) { /* process */ }
}
```

■ **Follow-up:** 'How is offsetsForTimes() used for disaster recovery?' After an incident, you can replay events from a specific point in time. For example, if a bug corrupted data from 2 PM to 3 PM, seek all consumers to 2 PM, fix the bug, and re-process. This is Kafka's killer feature vs traditional queues.

Q59. What is rack awareness in Kafka? How does it improve fault tolerance?

Medium Architecture

Answer:

Rack awareness ensures Kafka distributes replicas across different **failure domains** (racks, availability zones, data centers). Without it, all 3 replicas of a partition might end up on brokers in the same AZ — a single AZ failure loses all copies.

```
# Broker config: assign each broker to its rack/AZ
# broker-1 (us-east-1a):
broker.rack=us-east-1a

# broker-2 (us-east-1b):
broker.rack=us-east-1b

# broker-3 (us-east-1c):
broker.rack=us-east-1c

# With RF=3 and 3 racks, Kafka guarantees:
# Partition-0: leader=broker-1(1a), follower=broker-2(1b), follower=broker-3(1c)
# Each replica in a DIFFERENT AZ -> survives any single AZ failure

# Strimzi Kubernetes: automatic rack awareness
spec:
  kafka:
    rack:
      topologyKey: topology.kubernetes.io/zone  # Uses K8s node AZ label
```

■■ **Trap:** Without *broker.rack*, Kafka's replica assignment is rack-unaware. With 6 brokers in 2 AZs, it's possible for all 3 replicas to land in the same AZ. One AZ outage = data loss. Always configure rack awareness in multi-AZ deployments.

Q60. What is Tiered Storage in Kafka? How does it reduce storage costs?

Medium Architecture

Answer:

Tiered Storage (KIP-405, GA in Kafka 3.6+) allows Kafka to offload older log segments to cheap remote storage (S3, GCS, Azure Blob) while keeping recent data on local broker disks.

Aspect	Without Tiered Storage	With Tiered Storage
Retention 30d	All 30 days on broker local disk (expensive SSDs)	Last 1-2 days local; 28+ days on S3 (cheap)

Broker disk	Scales linearly with retention * throughput	Fixed size, independent of retention period
Cost (100TB)	~$10,000/mo (SSD)	~$500/mo (S3) + small local SSD
Scaling brokers	Must add brokers for more storage	Add brokers only for throughput, not storage
Replay old data	From local disk (fast)	From S3 (slightly slower first fetch, then cached)

```
# Enable tiered storage (Kafka 3.6+ broker config)
remote.log.storage.system.enable=true
remote.log.storage.manager.class.name=
    org.apache.kafka.server.log.remote.storage.RemoteLogManagerConfig

# S3 plugin config
rsm.config.s3.bucket.name=kafka-tiered-storage
rsm.config.s3.region=us-east-1

# Topic-level: enable tiered storage
kafka-configs.sh --alter --topic order-events \
  --add-config remote.storage.enable=true,\
  local.retention.ms=172800000,\          # 2 days local
  retention.ms=2592000000                  # 30 days total (28d on S3)
```

■ **Pro Tip:** Tiered Storage is a game-changer for cost: keep 30-90 day retention without massive broker disks. The local tier serves hot data (real-time consumers), while the remote tier handles replay and audit use cases. This makes Kafka viable as a long-term event store.

Q61. What is the difference between consumer.assign() and consumer.subscribe()?

Easy Consumer

Answer:

Aspect	subscribe()	assign()
Partition assignment	Automatic — group coordinator assigns partitions dynamically	Manual — you specify exact partitions to consume
Rebalancing	Yes — partitions redistributed when consumers join/leave	No — no consumer group coordination at all
Consumer group	Required (group.id must be set)	Optional (works without group)
Scaling	Automatic — add consumers, partitions redistribute	Manual — you manage which consumer reads which partition
Use case	Normal production consumers; scalable microservices	Tools, testing, specific partition replay, monitoring utilities

```
// subscribe(): automatic, scalable, standard approach
consumer.subscribe(List.of("order-events"));
// Group coordinator assigns partitions. Rebalances on changes.

// assign(): manual, full control, no rebalancing
consumer.assign(List.of(
    new TopicPartition("order-events", 0),
    new TopicPartition("order-events", 1)));
// YOU choose exactly which partitions. No group coordinator involved.

// CANNOT mix: subscribe() and assign() are mutually exclusive!
// consumer.subscribe(...); consumer.assign(...); // -> IllegalStateException
```

■■ **Trap:** You CANNOT use both *subscribe()* and *assign()* on the same consumer. They are mutually exclusive. *subscribe()* is for 99% of production use cases. Use *assign()* only for tools, testing, or specific-partition replay.

Q62. How do you implement priority processing with Kafka?

Hard Design Patterns

Answer:

Kafka doesn't have built-in message priority like JMS. Since partition ordering is strict FIFO, you need architectural patterns to achieve priority processing:

Pattern 1: Priority Topics (recommended)

```
// Separate topics by priority level
// order-events-high   -> 6 partitions, concurrency=6
// order-events-medium -> 6 partitions, concurrency=3
// order-events-low    -> 3 partitions, concurrency=1

// Producer routes by priority
public void send(Order order) {
    String topic = switch (order.getPriority()) {
        case HIGH   -> "order-events-high";
        case MEDIUM -> "order-events-medium";
        case LOW    -> "order-events-low";
    };
    kafkaTemplate.send(topic, order.getId(), order);
}

// Consumers: more resources allocated to high-priority topic
@KafkaListener(topics = "order-events-high", concurrency = "6")
public void highPriority(Order order) { processOrder(order); }

@KafkaListener(topics = "order-events-medium", concurrency = "3")
public void medPriority(Order order) { processOrder(order); }

@KafkaListener(topics = "order-events-low", concurrency = "1")
public void lowPriority(Order order) { processOrder(order); }
```

Pattern 2: Priority Partition (custom partitioner)

Route VIP messages to partition 0, dedicate a fast consumer to it. Standard messages go to partitions 1-N. Simpler but less flexible than separate topics.

■ **Pro Tip:** Priority Topics is the cleanest pattern: independent scaling, independent monitoring, independent DLTs. Allocate more consumer threads and pods to high-priority topics. SLA per topic, not per partition.

Q63. What is the Sticky Partitioner? How did Kafka 2.4 change null-key message routing?

Medium Producer

Answer:

Kafka Version	Null Key Behavior	Impact
Before 2.4 (RoundRobin)	Each record sent to a DIFFERENT partition in round-robin fashion	Tiny batches (1 record each). High overhead, poor throughput, many requests.
2.4+ (Sticky Partitioner)	All records in a batch sent to the SAME partition. Switches on batch completion.	Full batches, fewer requests, 2-3x better throughput. linger.ms more effective.

```
// Before Kafka 2.4 (null key, round-robin):
// Record-1 -> P0, Record-2 -> P1, Record-3 -> P2, Record-4 -> P0 ...
// Each partition gets 1 record per batch cycle -> tiny batches

// Kafka 2.4+ (null key, sticky partitioner):
// Record-1 -> P0, Record-2 -> P0, Record-3 -> P0 ... (batch fills up)
// Batch full -> switch to P1
// Record-N -> P1, Record-N+1 -> P1 ... (batch fills up)
// Result: full batches, much fewer network requests

// Explicitly set (default since 2.4):
props.put(ProducerConfig.PARTITIONER_CLASS_CONFIG,
    "org.apache.kafka.clients.producer.internals.DefaultPartitioner");
```

■ **Follow-up:** 'Does Sticky Partitioner cause hot partitions?' Over time, no — it distributes evenly across all partitions. It just fills batches to one partition at a time. Short-term imbalance is minimal and the throughput gain far outweighs it.

Q64. What is the difference between event sourcing and event-driven architecture?

Medium Design Patterns

Answer:

Aspect	Event-Driven Architecture	Event Sourcing
Core idea	Services communicate through asynchronous events instead of synchronous HTTP calls	Application state is derived by replaying a sequence of immutable domain events
Event role	Integration: events NOTIFY other services of changes	Storage: events ARE the primary source of truth
State storage	Traditional DB (MySQL, Postgres) + events for notification	Event log IS the database. No separate state table.

Replay	Events used for integration, not for state reconstruction	Replay ALL events from beginning to rebuild state
Kafka role	Message bus between services	Append-only event store (with compaction for snapshots)
Example	OrderService -> Kafka -> PaymentService reacts	OrderService stores: Created, ItemAdded, Shipped events. Current state = replay all.
Complexity	Moderate: eventual consistency, event ordering	High: event versioning, snapshots, projections

Key distinction: Event-Driven = services react to events (Kafka is the bus). Event Sourcing = events are the database (Kafka stores the truth). Most systems use Event-Driven. Event Sourcing is a specialized pattern for audit-heavy domains (finance, healthcare, legal).

■ **Follow-up:** 'Can Kafka be used as an Event Store?' Yes, with caveats. Use compacted topics for snapshots, long retention for event log. But Kafka lacks per-aggregate querying — you can't efficiently read 'all events for order-123' without scanning the partition. EventStoreDB or Axon are purpose-built alternatives.

Q65. How do you handle schema compatibility modes in Schema Registry?

Medium Schema Registry

Answer:

Mode	Rule	When to Use
BACKWARD (default)	New schema can READ data written by old schema. (new consumer, old producer)	Standard evolution: add fields with defaults, remove optional fields
FORWARD	Old schema can READ data written by new schema. (old consumer, new producer)	When consumers can't be updated before producers
FULL	Both BACKWARD and FORWARD. Both old and new schemas can read each other's data.	Safest. When producer and consumer deploy order is unpredictable.
NONE	No compatibility check. Any change allowed.	Development/testing only. NEVER in production.
BACKWARD_ TRANSITIVE	Backward compat checked against ALL previous versions, not just the latest.	When consumers may run very old schema versions (slow rollout teams)
FULL_ TRANSITIVE	Full compat against ALL previous versions.	Most strict. Enterprise environments with many teams sharing topics.

```
# Set compatibility mode per subject
curl -X PUT http://schema-registry:8081/config/order-events-value \
  -H "Content-Type: application/vnd.schemaregistry.v1+json" \
  -d '{"compatibility": "FULL_TRANSITIVE"}'

# Test compatibility before registering
curl -X POST http://schema-registry:8081/compatibility/subjects/\
  order-events-value/versions/latest \
  -H "Content-Type: application/vnd.schemaregistry.v1+json" \
  -d '{"schema": "..."}'
# Response: {"is_compatible": true} or {"is_compatible": false}
```

■ **Pro Tip:** Production recommendation: **FULL_TRANSITIVE** for shared topics across teams. It's the most restrictive but prevents any breaking change from reaching production. Teams can add fields with defaults and remove fields that had defaults — that's enough for 95% of schema evolution.

Q66. What is fetch.min.bytes and fetch.max.wait.ms? How do they affect consumer performance?

Medium Consumer Tuning

Answer:

Config	Default	What It Does	Effect
fetch.min.bytes	1 byte	Minimum data the broker must have before responding to a fetch request.	Higher = fewer fetch requests, fuller responses, higher throughput, added latency.

fetch.max.wait.ms	500 ms	Max time broker waits to accumulate fetch.min.bytes before responding.	Caps the latency introduced by fetch.min.bytes. Broker responds at this deadline.
max.partition.fetch.bytes	1048576 (1 MB)	Max data returned per partition per fetch.	Controls memory usage. Total fetch = this * number of assigned partitions.

```
// HIGH THROUGHPUT consumer: fuller fetches, batch processing
props.put("fetch.min.bytes", 65536);        // Wait for 64KB before responding
props.put("fetch.max.wait.ms", 1000);       // Max 1s wait for 64KB
props.put("max.poll.records", 500);         // Process 500 records per poll
props.put("max.partition.fetch.bytes", 2097152);  // 2MB per partition

// LOW LATENCY consumer: immediate response
props.put("fetch.min.bytes", 1);            // Respond with any data (default)
props.put("fetch.max.wait.ms", 100);        // Max 100ms wait
props.put("max.poll.records", 10);          // Small batches for fast processing
```

■ **Follow-up:** 'How do fetch.min.bytes and linger.ms relate?' They're mirrors: *linger.ms* batches on the producer side (outgoing), *fetch.min.bytes* batches on the consumer side (incoming). Tune both for optimal end-to-end throughput.

Q67. How do you implement multi-tenant Kafka? What are the isolation strategies?

Hard Architecture

Answer:

Strategy	Isolation	Cost	Complexity	When to Use
Shared cluster, shared topics	Low — noisy neighbor risk	Lowest — single cluster	Low — quotas + ACLs	Internal teams, trusted tenants, non-critical data
Shared cluster, separate topics	Medium — quotas + ACLs per tenant	Medium — single cluster, more topics	Medium — topic naming, ACL management	SaaS platforms, moderate isolation requirements
Separate clusters per tenant	High — full isolation, no shared resources	Highest — multiple clusters	High — cluster provisioning, management	Financial services, healthcare, strict data residency

```
// Pattern: Shared cluster, tenant-prefixed topics
// tenant-a.order-events  (12 partitions, RF=3)
// tenant-b.order-events  (6 partitions, RF=3)
// tenant-c.order-events  (3 partitions, RF=3)

// ACLs: each tenant can only access their own topics
kafka-acls.sh --add --allow-principal User:tenant-a \
  --operation Read --operation Write \
  --topic tenant-a. --resource-pattern-type prefixed

// Quotas: prevent one tenant from starving others
kafka-configs.sh --alter --add-config \
  'producer_byte_rate=10485760,consumer_byte_rate=20971520' \
  --entity-type users --entity-name tenant-a

// Spring Boot: tenant-aware producer
public void send(String tenantId, String key, String payload) {
    String topic = tenantId + ".order-events";
    kafkaTemplate.send(topic, key, payload);
}
```

■ **Pro Tip:** For most SaaS platforms: shared cluster + tenant-prefixed topics + ACLs + quotas. This balances cost and isolation. Reserve separate clusters only for tenants with strict compliance requirements (data residency, HIPAA, PCI-DSS).

Q68. What is the min.insync.replicas vs replication.factor relationship? Walk through failure scenarios.

Hard Architecture

Answer:

These two configs together define Kafka's **durability guarantee and availability trade-off**. Understanding their interaction under failure is critical.

Configuration: RF=3, min.insync.replicas=2, acks=all

```
// Setup: 3 brokers, topic with RF=3, min.insync.replicas=2

// Scenario 1: All 3 brokers healthy
// ISR = {broker-0, broker-1, broker-2}
// Write: acks=all waits for all 3 -> committed
// Result: STRONGEST guarantee. Can survive 2 broker failures for reads.

// Scenario 2: 1 broker dies
// ISR = {broker-0, broker-1}  (ISR size = 2, meets min.insync)
// Write: acks=all waits for 2 -> committed
// Result: Still available for reads AND writes. Can survive 1 more failure for reads.

// Scenario 3: 2 brokers die
// ISR = {broker-0}  (ISR size = 1, BELOW min.insync)
// Write: acks=all -> NOT ENOUGH REPLICAS EXCEPTION -> write REJECTED
// Read: Still possible (leader alive), but no new writes accepted
// Result: Read-only mode. Data protected from loss.

// Scenario 4: All 3 brokers die
// No leader available. Topic completely unavailable.
// Data safe on disk if disks are intact. Recovers when brokers restart.
```

RF	min.insync	acks	Brokers to lose before write fails	Brokers to lose before data loss
3	2	all	2 (3rd alone can't meet min)	3 (all copies must be lost)
3	1	all	All 3 (any 1 broker suffices)	3 (but risk: if only 1 has data and it dies)
3	2	1	Never (only leader needed)	1 (if leader dies before replication)
2	1	all	2 (same as RF=1)	2 (only 2 copies)

■■ **Trap:** The #1 production mistake: setting *min.insync.replicas=2* on the broker but using *acks=1* on the producer. This renders min.insync USELESS — acks=1 only waits for the leader. Always verify both sides: broker min.insync AND producer acks=all.

■ This completes **Part 4 (Questions 52–68)** covering Windowed Operations, Exactly-Once Streams, Saga Pattern, CQRS, CDC/Debezium, AdminClient API, Consumer Seek, Rack Awareness, Tiered Storage, assign() vs subscribe(), Priority Processing, Sticky Partitioner, Event Sourcing vs EDA, Schema Compatibility, Consumer Tuning, Multi-Tenancy, and Replication Failure Scenarios. **Part 5** will cover Questions 69–85. Say **'next'** to continue!

100 Kafka Interview

Questions & Answers

with Detailed Code Examples

Part 5: Questions 69 – 85

SMTs • State Stores • Compression • Timestamps • Rebalance Listeners

Unclean Election • @SendTo • Punctuators • Controller Failover • Topic Configs

by **TechTribers**

@techtribes • www.techtriber.com

Q69. What are Single Message Transforms (SMTs) in Kafka Connect? Give practical examples.

Medium Kafka Connect

Answer:

SMTs are **lightweight, configurable transformations** applied per-record inside a Kafka Connect pipeline. They modify records in-flight without writing custom code.

SMT	What It Does	Example Use Case
ExtractField	Extracts a single field from key or value	Pull orderId from nested JSON as the record key
MaskField	Replaces field with type-valid default (0, "", null)	Mask PII fields (email, phone) before sending to analytics
TimestampRouter	Modifies topic name based on record timestamp	Route to daily topics: orders-2024-01-15
RegexRouter	Rewrites topic name using regex substitution	Rename CDC topics: cdc.public.orders → orders
ValueToKey	Copies fields from value into the record key	Set orderId as key for partition-based ordering
InsertHeader	Adds a header to every record	Inject sourceCluster header for multi-DC routing
ExtractNewRecordState (Debezium)	Flattens Debezium envelope to just the after state	Simplify CDC records for downstream consumers

```
// Kafka Connect config: chain multiple SMTs
{
  "name": "pg-cdc-to-elastic",
  "config": {
    "connector.class": "io.debezium.connector.postgresql.PostgresConnector",
    "transforms": "unwrap,route,mask",

    "transforms.unwrap.type": "io.debezium.transforms.ExtractNewRecordState",
    "transforms.unwrap.drop.tombstones": "false",

    "transforms.route.type": "org.apache.kafka.connect.transforms.RegexRouter",
    "transforms.route.regex": "cdc\\.public\\.(.*)",
    "transforms.route.replacement": "$1",

    "transforms.mask.type": "org.apache.kafka.connect.transforms.MaskField$Value",
    "transforms.mask.fields": "email,phone,ssn"
  }
}
// Pipeline: PG WAL -> Debezium -> unwrap envelope -> rename topic -> mask PII -> Kafka
```

■ **Pro Tip:** SMTs are applied in ORDER of the comma-separated list. Put structural transforms (unwrap) first, routing (RegexRouter) second, and field-level transforms (mask) last. The order matters because each SMT sees the output of the previous one.

Q70. What are Kafka Streams state stores? How do interactive queries work?

Hard Kafka Streams

Answer:

State stores are **local, embedded databases** (RocksDB by default) that Kafka Streams uses to maintain aggregation results, join state, and windowed computations. They're backed by Kafka changelog topics for fault tolerance.

Store Type	Backed By	Use Case
Persistent (RocksDB)	On-disk RocksDB + Kafka changelog topic	Production aggregations, joins, windows. Survives restarts.
In-memory	Heap memory + Kafka changelog topic	Small datasets, low-latency. Full rebuild on restart.
Custom	Your implementation + Kafka changelog topic	Redis, external DB as state store for special requirements

Interactive Queries: Query State Store via REST

```
// Kafka Streams topology with named state store
KTable<String, Long> orderCounts = orders
    .groupByKey()
    .count(Materialized.<String, Long, KeyValueStore<Bytes, byte[]>>
        as("order-count-store"));  // Named store for interactive queries

// REST endpoint to query the state store
@RestController @RequiredArgsConstructor
public class OrderCountController {
    private final StreamsBuilderFactoryBean streamsFactory;

    @GetMapping("/api/orders/count/{userId}")
    public ResponseEntity<Long> getCount(@PathVariable String userId) {
        KafkaStreams streams = streamsFactory.getKafkaStreams();
        ReadOnlyKeyValueStore<String, Long> store = streams.store(
            StoreQueryParameters.fromNameAndType(
                "order-count-store", QueryableStoreTypes.keyValueStore()));
        Long count = store.get(userId);
        return count != null
            ? ResponseEntity.ok(count)
            : ResponseEntity.notFound().build();
    }
}
// GET /api/orders/count/user-123 -> 42 (real-time, from local RocksDB)
```

■ **Follow-up:** 'What if the key is on a different instance?' Each instance only has state for its assigned partitions. Use *streams.metadataForKey()* to find which instance hosts the key, then forward the REST request to that instance. Libraries like *kafka-streams-interactive-queries* handle this routing.

Q71. Compare Kafka compression types: none, gzip, snappy, lz4, and zstd.

Medium Performance

Answer:

Type	Compression Ratio	CPU Speed	Memory	Best For
none	1.0x (no compression)	Zero CPU overhead	None	Ultra-low latency; tiny payloads where compression wastes CPU
gzip	Best (60-80% reduction)	Slowest (~20MB/s)	High	Archival, infrequent reads, bandwidth-constrained networks
snappy	Moderate (50-60%)	Fast (~250MB/s)	Low	Legacy systems; balanced CPU/ratio. Popular older default.
lz4	Moderate (50-65%)	Fastest (~400MB/s)	Low	Most production systems. Best speed-to-ratio. RECOMMENDED default.
zstd	Near gzip (60-75%)	Good (~150MB/s)	Medium	When ratio matters more than CPU. Large payloads, high-volume.

```
// Producer config — applied at BATCH level (not per-record)
props.put(ProducerConfig.COMPRESSION_TYPE_CONFIG, "lz4");

// Broker config — override producer compression (optional)
compression.type=producer  // Default: use producer's compression (recommended)
// OR: compression.type=lz4  // Force re-compress all data

// Topic-level override:
kafka-configs.sh --alter --topic order-events \
  --add-config compression.type=zstd
```

■■ **Trap:** Compression happens at the BATCH level, not per-record. With *linger.ms=0* and small batches (1-2 records), compression overhead exceeds savings. Always pair compression with *linger.ms=5-20* and reasonable *batch.size* for maximum benefit.

Q72. What is unclean.leader.election.enable? Why is it the most dangerous Kafka config?

Hard Architecture

Answer:

When all ISR replicas for a partition are offline and *unclean.leader.election.enable=true*, Kafka promotes a **non-ISR (out-of-sync) replica** as leader. This replica is guaranteed to have **missing data** — causing permanent, irrecoverable data loss.

Setting	Behavior on Total ISR Loss	Trade-off

false (default since Kafka 0.11)	Partition goes OFFLINE. No reads or writes until an ISR member recovers.	Availability sacrificed for data safety. No data loss.
true	Out-of-sync replica becomes leader. Missing records are PERMANENTLY LOST.	Data safety sacrificed for availability. Irrecoverable loss.

```
// Scenario: RF=3, ISR={broker-0, broker-1, broker-2}
// broker-0: offset 1000 (leader, fully caught up)
// broker-1: offset 1000 (ISR, fully caught up)
// broker-2: offset 950  (lagging, NOT in ISR, 50 records behind)

// ALL ISR members (broker-0, broker-1) crash simultaneously

// unclean.leader.election.enable=false:
//   Partition OFFLINE. Waits for broker-0 or broker-1 to recover.
//   No data loss. Applications get errors until recovery.

// unclean.leader.election.enable=true:
//   Broker-2 (offset 950) becomes leader.
//   Records 951-1000 are PERMANENTLY LOST.
//   New writes start at offset 951. Old data is GONE FOREVER.
//   Consumer that committed offset 990 now reads corrupt state.

# ALWAYS set to false in production:
unclean.leader.election.enable=false
```

■■ **Trap:** This is the **one config that can cause permanent, irrecoverable data loss**. Even with acks=all and min.insync.replicas=2, if ALL ISR brokers crash and unclean election is enabled, you lose data. The default (false) is the only safe choice for production.

Q73. What are the two types of timestamps in Kafka records? How do they affect processing?

Medium Internals

Answer:

Timestamp Type	Config Value	Who Sets It	When to Use
CreateTime (default)	message.timestamp.type=CreateTime	Producer sets at send time (or API assigns current time)	Event time semantics. Timestamp = when event actually occurred.
LogAppendTime	message.timestamp.type=LogAppendTime	Broker overwrites with broker clock at append time	Ingest time semantics. Useful when producer clocks are unreliable.

```
// Producer: set explicit timestamp (CreateTime mode)
ProducerRecord<String, String> record = new ProducerRecord<>(
    "order-events",    // topic
    0,                 // partition (null for auto)
    event.getOccurredAt().toEpochMilli(),  // EXPLICIT timestamp
    orderId,           // key
    payload);          // value

// Consumer: read the timestamp
for (ConsumerRecord<String, String> r : records) {
    System.out.printf("offset=%d timestamp=%d type=%s%n",
        r.offset(), r.timestamp(), r.timestampType());
    // timestampType() returns CREATE_TIME or LOG_APPEND_TIME
}

// Kafka Streams: timestamp extraction for windowing
props.put(StreamsConfig.DEFAULT_TIMESTAMP_EXTRACTOR_CLASS_CONFIG,
    WallclockTimestampExtractor.class);  // Or custom extractor

// Time-based seek uses this timestamp:
consumer.offsetsForTimes(Map.of(tp, targetTimestamp));
```

■ **Follow-up:** 'When does LogAppendTime break things?' Kafka Streams windowing and *offsetsForTimes()* rely on record timestamps. With LogAppendTime, timestamps reflect ingest time, NOT event time. If events arrive late, they're windowed incorrectly. Use CreateTime with an explicit event timestamp for stream processing.

Q74. What is buffer.memory and max.block.ms in the Kafka producer?

Medium Producer Internals

Answer:

Config	Default	What It Does
buffer.memory	33554432 (32 MB)	Total memory the producer allocates for buffering records waiting to be sent. Shared across all partitions. When full, send() blocks.

max.block.ms	60000 (60 sec)	Max time send() blocks when buffer.memory is full or metadata is unavailable. After this, throws TimeoutException.

What Happens Under Pressure

- **Normal:** *send()* adds record to the RecordAccumulator (batch buffer). Returns immediately with Future.
- **Buffer filling:** Producer sends faster than network can drain. Buffer usage grows toward 32MB.
- **Buffer full:** *send()* BLOCKS the calling thread. Waits for space (in-flight batches to complete).
- **Timeout:** If blocked for > *max.block.ms* (60s), throws *TimeoutException*. Application must handle.

```
// High-throughput producer: larger buffer, shorter timeout
props.put(ProducerConfig.BUFFER_MEMORY_CONFIG, 67108864);  // 64MB
props.put(ProducerConfig.MAX_BLOCK_MS_CONFIG, 10000);      // 10s fail-fast

// Handling the timeout in application code:
try {
    producer.send(record, callback);
} catch (TimeoutException e) {
    // Buffer full for > 10s. Options:
    // 1. Log and drop (metrics/non-critical)
    // 2. Write to local fallback (file, DB queue)
    // 3. Throw to caller for circuit breaker handling
    log.error("Kafka buffer full, cannot send: {}", e.getMessage());
    fallbackQueue.add(record);
}
```

■ **Pro Tip:** If you see *TimeoutException* from *send()*, your producer is overwhelmed. Diagnostics: (1) Check if broker is slow (network/disk). (2) Increase *buffer.memory*. (3) Tune *batch.size* + *linger.ms*. (4) Add more brokers/partitions. (5) Check compression is enabled.

Q75. What are ConsumerRebalanceListener callbacks? How do you use them?

Medium Consumer

Answer:

ConsumerRebalanceListener provides two callbacks that fire during consumer group rebalancing: **onPartitionsRevoked** (before partitions taken away) and **onPartitionsAssigned** (after new partitions given).

```
consumer.subscribe(List.of("order-events"), new ConsumerRebalanceListener() {

    @Override
    public void onPartitionsRevoked(Collection<TopicPartition> partitions) {
        // Called BEFORE partitions are revoked. Last chance to:
        // 1. Commit current offsets (avoid reprocessing)
        // 2. Flush any buffered data
        // 3. Close resources tied to these partitions
        log.info("Revoking: {}", partitions);
        consumer.commitSync();           // Commit before losing partitions
        flushBatchBuffer(partitions);    // Flush any in-memory batch
    }

    @Override
    public void onPartitionsAssigned(Collection<TopicPartition> partitions) {
        // Called AFTER new partitions are assigned. Initialize:
        // 1. Load state for new partitions
        // 2. Seek to specific offset if needed
        // 3. Initialize partition-specific resources
        log.info("Assigned: {}", partitions);
        for (TopicPartition tp : partitions) {
            Long savedOffset = offsetStore.get(tp);
            if (savedOffset != null)
                consumer.seek(tp, savedOffset);  // Resume from external store
        }
    }
});

// Spring Kafka equivalent:
@KafkaListener(topics = "order-events")
public class OrderConsumer implements ConsumerSeekAware {
    @Override
    public void onPartitionsAssigned(
            Map<TopicPartition, Long> assignments, ConsumerSeekCallback callback) {
        // Custom seek logic on assignment
    }
}
```

■ **Follow-up:** 'Why commit in onPartitionsRevoked?' Without it, any un-committed progress is lost. Another consumer gets the partition and re-processes from the last committed offset. Always commit synchronously (not async) in revocation to guarantee progress is saved.

Q76. What is Kafka Streams punctuate()? How do you implement scheduled operations in stream processing?

Hard Kafka Streams

Answer:

Punctuators are scheduled callbacks in Kafka Streams that fire at regular intervals, allowing periodic operations like flushing aggregations, emitting alerts, or cleaning expired state.

Punctuate Type	Triggered By	Use Case

WALL_CLOCK_TIME	System clock (real time). Fires even if no records arrive.	Periodic health checks, emit heartbeats, timeout detection
STREAM_TIME	Record timestamps (event time). Only advances when records are processed.	Emit windowed results, expire state entries, periodic aggregation flushes

```
// Processor API: detect inactive users every 60 seconds
public class InactivityDetector implements Processor<String, UserEvent, String, Alert> {
    private KeyValueStore<String, Long> lastSeenStore;
    private ProcessorContext<String, Alert> context;

    @Override
    public void init(ProcessorContext<String, Alert> ctx) {
        this.context = ctx;
        this.lastSeenStore = ctx.getStateStore("last-seen-store");

        // Schedule punctuator every 60 seconds (wall clock)
        ctx.schedule(Duration.ofSeconds(60), PunctuationType.WALL_CLOCK_TIME,
            timestamp -> {
                long threshold = timestamp - Duration.ofMinutes(30).toMillis();
                try (var iter = lastSeenStore.all()) {
                    while (iter.hasNext()) {
                        var entry = iter.next();
                        if (entry.value < threshold) {
                            context.forward(new Record<>(entry.key,
                                new Alert("User inactive: " + entry.key),
                                timestamp));
                            lastSeenStore.delete(entry.key);
                        }
                    }
                }
            });
    }

    @Override
    public void process(Record<String, UserEvent> record) {
        lastSeenStore.put(record.key(), record.timestamp());
    }
}
```

■■ **Trap:** STREAM_TIME punctuators ONLY fire when records arrive. If no data flows, the punctuator never triggers — making it useless for timeout detection. Use WALL_CLOCK_TIME for inactivity/timeout scenarios.

Q77. What is the @SendTo annotation in Spring Kafka? How do you chain topic processing?

Medium Spring Boot

Answer:

@SendTo automatically forwards the return value of a *@KafkaListener* method to another Kafka topic, enabling **declarative topic chaining** without manual KafkaTemplate usage.

```
// Basic chaining: order-events -> enriched-orders
@KafkaListener(topics = "order-events")
@SendTo("enriched-orders")
public EnrichedOrder processOrder(OrderEvent event) {
    UserProfile user = userService.getUser(event.getUserId());
    return EnrichedOrder.builder()
        .orderId(event.getOrderId())
        .userName(user.getName())
        .loyaltyTier(user.getTier())
        .amount(event.getAmount())
        .build();
    // Return value automatically sent to "enriched-orders" topic
}

// Dynamic routing: return value determines target topic
@KafkaListener(topics = "raw-events")
@SendTo  // No topic specified — uses Message headers
public Message<?> route(RawEvent event) {
    String targetTopic = event.getType().equals("ORDER")
        ? "order-events" : "misc-events";
    return MessageBuilder.withPayload(processEvent(event))
        .setHeader(KafkaHeaders.TOPIC, targetTopic)
        .setHeader(KafkaHeaders.KEY, event.getId())
        .build();
}

// Conditional forwarding: return null to skip
@KafkaListener(topics = "all-events")
@SendTo("important-events")
public ImportantEvent filter(Event event) {
    if (event.getPriority() < 5) return null;  // null = don't forward
    return new ImportantEvent(event);
}
```

■ **Pro Tip:** @SendTo is perfect for simple pipelines: filter, transform, route. For complex multi-step processing with state, prefer Kafka Streams. @SendTo is stateless per-record transformation; Streams is stateful stream processing.

Q78. What is the difference between topic-level and broker-level configurations?

Easy Operations

Answer:

Config	Broker-Level (server.properties)	Topic-Level Override
Retention time	log.retention.hours=168 (7 days) Applies to all topics by default	retention.ms=2592000000 (30 days) Per-topic override
Partition count	num.partitions=12 (auto-create default)	Set at creation time. Can increase, never decrease.
Replication factor	default.replication.factor=3	Set at creation time. Can change via reassignment.
Cleanup policy	log.cleanup.policy=delete	cleanup.policy=compact (per-topic)
Min ISR	min.insync.replicas=2	min.insync.replicas=3 (stricter per-topic)
Compression	compression.type=producer	compression.type=zstd (force per-topic)
Max message	message.max.bytes=1048576 (1MB)	max.message.bytes=10485760 (10MB per-topic)

```
# View topic-level overrides:
kafka-configs.sh --describe --topic order-events \
  --bootstrap-server broker:9092
# Shows: Dynamic configs for topic 'order-events' are:
#   retention.ms=2592000000

# Set topic-level config:
kafka-configs.sh --alter --topic order-events \
  --add-config retention.ms=2592000000,min.insync.replicas=2 \
  --bootstrap-server broker:9092

# Spring Boot: NewTopic bean with topic-level configs
@Bean
public NewTopic auditTopic() {
    return TopicBuilder.name("audit-events")
        .partitions(12).replicas(3)
        .config("retention.ms", "-1")               // Infinite retention
        .config("cleanup.policy", "compact,delete")  // Compact + delete
        .config("min.insync.replicas", "2")
        .build();
}
```

■ **Follow-up:** 'Which takes precedence?' Topic-level always overrides broker-level for the same config. Broker-level is the default for topics that don't have explicit overrides. This lets you set safe defaults globally and customize per-topic.

Q79. How does Kafka controller failover work? What happens during a controller election?

Hard Architecture

Answer:

ZooKeeper Mode (legacy)

- **1.** The current controller broker crashes or loses ZK session.
- **2.** ZK's ephemeral */controller* znode disappears.
- **3.** All brokers have a watch on */controller*. They race to create a new ephemeral znode.
- **4.** Winner becomes new controller. Reads ALL metadata from ZK (topics, partitions, ISR).
- **5.** New controller performs leader election for any leaderless partitions.
- **6.** Broadcasts updated metadata to all brokers. Failover: **30s–120s** depending on cluster size.

KRaft Mode (modern)

- **1.** Active controller in the quorum fails.
- **2.** Remaining quorum members run Raft leader election (majority vote).
- **3.** New leader already has all metadata (replicated via __cluster_metadata topic).
- **4.** No full state read required. Failover: **1–5 seconds**.

```
# KRaft: controller quorum is 3 nodes
controller.quorum.voters=1@ctrl-1:9093,2@ctrl-2:9093,3@ctrl-3:9093

# If ctrl-1 (active controller) fails:
# 1. ctrl-2 and ctrl-3 detect missing heartbeat
# 2. One calls election, other votes -> majority (2/3) achieved
# 3. New controller takes over in seconds
# 4. All metadata already replicated via __cluster_metadata Raft log

# ZK mode failover was slow because:
# - Full metadata read from ZK (tens of thousands of znodes)
# - More partitions = slower recovery (O(partitions))
# KRaft: metadata is a replicated log = O(1) failover
```

■ **Pro Tip:** This is a common architecture question. Key differentiator: ZK mode = full state rebuild on failover (minutes). KRaft = Raft log already replicated (seconds). Mention the O(partitions) vs O(1) complexity to show deep understanding.

Q80. What is the RecordAccumulator in the Kafka producer? How does the internal batching work?

Hard Producer Internals

Answer:

The **RecordAccumulator** is the producer's internal buffer that groups records into batches per partition. It's the core component that enables Kafka's high-throughput batching.

Internal Flow

- **1. send():** Application calls *send(record)*. Record is serialized and routed to a partition.
- **2. Append to Deque:** RecordAccumulator maintains a *Map<TopicPartition, Deque<ProducerBatch>>*. Record appended to the last batch in the deque for that partition.
- **3. Batch creation:** If no open batch exists or current batch is full (*batch.size*), a new *ProducerBatch* is allocated from *buffer.memory*.
- **4. Sender thread:** Background *Sender* thread wakes up when any batch is full OR *linger.ms* elapses. Drains ready batches and sends to brokers.
- **5. Compression:** Entire batch is compressed (not per-record). This is why larger batches compress better.

```
// Memory layout:
// buffer.memory = 32MB (total budget)
//
// RecordAccumulator {
//   partition-0: [Batch(32KB, full) -> Batch(12KB, open)]
//   partition-1: [Batch(32KB, full) -> Batch(32KB, full) -> Batch(5KB, open)]
//   partition-2: [Batch(8KB, open)]
// }
//
// Sender thread drains FULL batches:
//   -> Groups by broker (batches for same broker go in one request)
//   -> Sends ProduceRequest to each broker
//   -> Handles acks/retries per batch

// Key config interactions:
// batch.size = 32768     // Max bytes per batch
// linger.ms = 10         // Max wait before sending incomplete batch
// buffer.memory = 32MB   // Total memory for ALL batches across ALL partitions
// max.in.flight = 5      // Max unacked batches per broker connection
```

■ **Follow-up:** 'Why does Kafka batch per-partition, not per-broker?' Because each batch becomes a single entry in the partition's log segment. Per-partition batching enables: sequential writes, batch-level compression, and simpler offset assignment (batch offset = first record's offset).

Q81. How do you implement request-reply (synchronous) communication over Kafka?

Medium Design Patterns

Answer:

Although Kafka is designed for async messaging, **ReplyingKafkaTemplate** in Spring Kafka enables synchronous request-reply over Kafka topics. The producer sends a request and blocks until the consumer replies on a dedicated reply topic.

```
// 1. Configuration
@Bean
public ReplyingKafkaTemplate<String, String, String> replyingTemplate(
        ProducerFactory<String, String> pf,
        ConcurrentMessageListenerContainer<String, String> replyContainer) {
    return new ReplyingKafkaTemplate<>(pf, replyContainer);
}

@Bean
public ConcurrentMessageListenerContainer<String, String> replyContainer(
        ConsumerFactory<String, String> cf) {
    ContainerProperties props = new ContainerProperties("reply-topic");
    return new ConcurrentMessageListenerContainer<>(cf, props);
}

// 2. Producer side — send request, wait for reply
@Service @RequiredArgsConstructor
public class OrderValidator {
    private final ReplyingKafkaTemplate<String, String> template;

    public String validate(String orderId) throws Exception {
        ProducerRecord<String, String> record =
            new ProducerRecord<>("validation-requests", orderId, orderPayload);
        RequestReplyFuture<String, String, String> future =
            template.sendAndReceive(record, Duration.ofSeconds(10));
        ConsumerRecord<String, String> reply = future.get();
        return reply.value();  // Response from consumer
    }
}

// 3. Consumer side — process and reply
@KafkaListener(topics = "validation-requests")
@SendTo  // Replies to the topic specified in REPLY_TOPIC header
public String handleValidation(String request) {
    boolean valid = validationService.validate(request);
    return valid ? "VALID" : "INVALID";  // Sent back to reply topic
}
```

■■ **Trap:** Request-reply over Kafka adds latency (two topic hops vs one HTTP call) and complexity. Use it when you need Kafka's durability/ordering for the request, or when the consumer is behind Kafka and not directly addressable. For simple service-to-service calls, prefer HTTP/gRPC.

Q82. What are Kafka Connect converters? Explain the role of JsonConverter, AvroConverter, and ProtobufConverter.

Medium Kafka Connect

Answer:

Converters handle **serialization/deserialization** of Kafka Connect records between the Connect internal data format and the wire format stored in Kafka topics.

Converter	Wire Format	Schema	When to Use
JsonConverter (schemas.enable=true)	JSON with embedded schema in every record	Schema in every record (large payloads)	Development, debugging. Human-readable but large on disk.
JsonConverter (schemas.enable=false)	Schemaless JSON	No schema enforcement	Simple pipelines, when consumer handles schema internally
AvroConverter	Avro binary + Schema Registry schema ID	Schema Registry enforced; compact payloads	Production standard. Small payloads + schema evolution governance.
ProtobufConverter	Protobuf binary + Schema Registry	Schema Registry enforced; requires .proto files	When ecosystem already uses Protobuf extensively

```
// Kafka Connect worker config: set converter globally
key.converter=org.apache.kafka.connect.storage.StringConverter
value.converter=io.confluent.connect.avro.AvroConverter
value.converter.schema.registry.url=http://schema-registry:8081

// Per-connector override (overrides worker default):
{
  "name": "my-connector",
  "config": {
    "value.converter": "org.apache.kafka.connect.json.JsonConverter",
    "value.converter.schemas.enable": "false"
  }
}
```

■ **Follow-up:** 'Should I use schemas.enable=true with JsonConverter?' Only for dev/debugging. It embeds the full schema in EVERY record, doubling payload size. For production, use AvroConverter with Schema Registry — schema stored once in Registry, only 5-byte ID in each record.

Q83. How do you configure Spring Kafka for transactional producers and exactly-once consumers?

Hard Spring Boot

Answer:

```
# application.yml — Full transactional setup
spring:
  kafka:
    bootstrap-servers: localhost:9092
    producer:
      transaction-id-prefix: order-txn-  # Enables transactional producer!
      acks: all
      enable-idempotence: true
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer: org.apache.kafka.common.serialization.StringSerializer
    consumer:
      group-id: order-txn-group
      enable-auto-commit: false
      isolation-level: read_committed   # Only see committed transactional records
      auto-offset-reset: earliest
    listener:
      ack-mode: record

---
// Transactional producer — sends multiple records atomically
@Service @RequiredArgsConstructor
public class TransactionalOrderService {
    private final KafkaTemplate<String, String> kafkaTemplate;

    // executeInTransaction wraps everything in a Kafka transaction
    public void processOrder(Order order) {
        kafkaTemplate.executeInTransaction(ops -> {
            ops.send("order-events", order.getId(), order.toJson());
            ops.send("audit-log", order.getId(), auditEntry(order));
            ops.send("notification-events", order.getUserId(), notify(order));
            // All 3 sends commit atomically. If any fails, ALL abort.
            return null;
        });
    }
}

// Consumer with read_committed — NEVER sees aborted records
@KafkaListener(topics = "order-events", groupId = "order-txn-group")
public void consume(OrderEvent event, Acknowledgment ack) {
    // This listener ONLY receives committed transactional records.
    // Aborted transactions are invisible.
    orderService.process(event);
    ack.acknowledge();
}
```

■ **Pro Tip:** The key Spring config: *transaction-id-prefix* on the producer side (enables transactions) and *isolation-level: read_committed* on the consumer side (filters aborted records). Without read_committed, consumers see ALL records including aborted ones.

Q84. What is the Kafka Streams DSL vs Processor API? When would you choose each?

Medium Kafka Streams

Answer:

Aspect	Streams DSL	Processor API
Abstraction	High-level, functional. map, filter, groupBy, join, aggregate, window.	Low-level, imperative. Full control over processing, state stores, scheduling.
Ease of use	Easy — declarative chaining. Less code, readable.	Complex — implement Processor/Transformer interfaces manually.
State management	Automatic via Materialized. State stores created by DSL.	Manual. You create, register, and manage state stores.
Scheduling	No scheduled operations. Only event-driven processing.	Punctuators for periodic operations (wall clock or stream time).
Flexibility	Limited to DSL operations. Custom logic via mapValues.	Full control. Access to ProcessorContext, forward, commit, state stores.
When to use	80% of use cases: filtering, aggregation, joins, windowing.	Complex event processing, custom timers, multi-step stateful transformations.

```
// DSL: clean, declarative (covers most use cases)
KStream<String, Order> orders = builder.stream("orders");
orders.filter((k, v) -> v.getAmount() > 100)
      .mapValues(v -> new EnrichedOrder(v, lookupUser(v.getUserId())))
      .groupByKey()
      .count(Materialized.as("order-count"))
      .toStream()
      .to("order-counts");

// Processor API: full control (complex scenarios)
builder.addStateStore(
    Stores.keyValueStoreBuilder(
        Stores.persistentKeyValueStore("dedup-store"),
        Serdes.String(), Serdes.Long()));

builder.stream("raw-events")
    .process(() -> new DeduplicationProcessor(), "dedup-store");
// DeduplicationProcessor: manual state store access, punctuators, custom forwarding
```

■ **Follow-up:** 'Can you mix DSL and Processor API?' Yes! Use *KStream.process()* or *KStream.transform()* to drop into the Processor API from a DSL topology. Common pattern: DSL for the pipeline, Processor API for one complex step.

Q85. Design a real-time fraud detection system using Kafka. Walk through the architecture.

Hard System Design

Answer:

Architecture Overview

- **Source:** Payment gateway publishes *TransactionEvent* to *payment-transactions* topic (Avro + Schema Registry).
- **Enrichment:** Kafka Streams joins transaction stream with *user-profiles* KTable (account age, country, limits).
- **Rule engine:** Windowed aggregations detect anomalies: > 5 transactions in 1 min, total > $10K in 1 hour, new device + high amount.
- **ML scoring:** Enriched transactions sent to *fraud-scoring* topic. ML service scores risk (0-100).
- **Decision:** If risk > 80 → block transaction, publish to *fraud-alerts*. If 50-80 → flag for review.
- **State store:** RocksDB maintains per-user state: recent transaction history, velocity counters, device fingerprints.

```
// Kafka Streams Topology for Fraud Detection
StreamsBuilder builder = new StreamsBuilder();

KStream<String, Transaction> txns = builder.stream("payment-transactions");
KTable<String, UserProfile> users = builder.table("user-profiles");

// Step 1: Enrich with user profile
KStream<String, EnrichedTxn> enriched = txns.join(users,
    (txn, user) -> new EnrichedTxn(txn, user));

// Step 2: Velocity check — count per user in 1-minute window
KTable<Windowed<String>, Long> velocityCounts = enriched
    .groupByKey()
    .windowedBy(TimeWindows.ofSizeWithNoGrace(Duration.ofMinutes(1)))
    .count(Materialized.as("velocity-store"));

// Step 3: Amount check — sum per user in 1-hour window
KTable<Windowed<String>, Double> hourlyAmounts = enriched
    .groupByKey()
    .windowedBy(TimeWindows.ofSizeWithNoGrace(Duration.ofHours(1)))
    .aggregate(() -> 0.0, (k, v, agg) -> agg + v.getAmount(),
        Materialized.as("hourly-amount-store"));

// Step 4: Rule-based filtering
enriched.filter((key, txn) ->
        txn.getAmount() > txn.getUser().getDailyLimit() * 0.5
        || txn.isNewDevice()
        || txn.getCountry() != txn.getUser().getHomeCountry())
    .to("fraud-scoring");    // Send to ML service for scoring

// Step 5: Alert on confirmed fraud
KStream<String, ScoredTxn> scored = builder.stream("fraud-scored");
scored.filter((k, v) -> v.getRiskScore() > 80)
      .to("fraud-alerts");  // Blocked transactions
scored.filter((k, v) -> v.getRiskScore() >= 50 && v.getRiskScore() <= 80)
      .to("fraud-review");  // Manual review queue
```

■ **Pro Tip:** This is a **common system design interview question**. Key points to mention: (1) Sub-second latency via Kafka Streams. (2) Windowed aggregations for velocity/amount rules. (3) Stream-table join for user context. (4) Interactive queries for real-time dashboards. (5) DLT for failed scoring calls.

■ This completes **Part 5 (Questions 69–85)** covering SMTs, State Stores & Interactive Queries, Compression Types, Unclean Leader Election, Record Timestamps, buffer.memory/max.block.ms, Rebalance Listeners, Punctuators, @SendTo Chaining, Topic vs Broker Configs, Controller Failover, RecordAccumulator Internals, Request-Reply Pattern, Connect Converters, Transactional Spring Kafka, DSL vs Processor API, and Fraud Detection System Design. **Part 6 (final)** will cover Questions 86–100. Say **'next'** to continue!

100 Kafka Interview

Questions & Answers

with Detailed Code Examples

Part 6 (Final): Questions 86 – 100

Tombstones • Leader Epoch • Follower Fetching • Metrics Interceptor • Graceful Shutdown

Multi-DC Design • Data Pipeline • Capacity Planning • Upgrade Strategies • Debugging

by **TechTribers**

@techtribes • www.techtriber.com

Q86. What are tombstone records in Kafka? How do they work with log compaction?

Medium Internals

Answer:

A **tombstone** is a record with a valid key but a **null value**. In compacted topics, tombstones signal that the key should be deleted during the next compaction cycle.

Tombstone Lifecycle

- **1. Published:** Producer sends record with key=X, value=null.
- **2. Retained:** Tombstone is kept in the log for *delete.retention.ms* (default 24 hours). During this window, consumers can read the tombstone and know key X was deleted.
- **3. Compaction:** After *delete.retention.ms*, the next compaction cycle removes the tombstone AND all previous values for key X. Key is permanently gone.

```
// Send a tombstone to delete a user from a compacted topic
producer.send(new ProducerRecord<>("user-profiles", userId, null));
// key = userId, value = null -> TOMBSTONE

// KTable in Kafka Streams: tombstone removes key from table
KTable<String, UserProfile> users = builder.table("user-profiles");
// When a tombstone arrives for userId="user-123":
// users.get("user-123") returns null
// Stream-table joins with "user-123" produce no match

// Config for tombstone retention (topic-level):
// delete.retention.ms=86400000  (24h default, how long tombstones live)
// min.compaction.lag.ms=0       (min time before record eligible for compaction)

// WARNING: If consumer reads AFTER tombstone is compacted away,
// it will never know the key existed or was deleted.
```

■ **Follow-up:** 'How does Kafka Streams handle tombstones in joins?' In a KStream-KTable join, a tombstone in the KTable removes the entry. Future stream records for that key produce no join result. In KStream-KStream joins, null values are skipped entirely.

Q87. What is the leader epoch? How does it prevent log divergence after failover?

Hard Internals

Answer:

The **leader epoch** is a monotonically increasing integer assigned each time a new leader is elected for a partition. It prevents **log divergence** — where a follower has records the new leader doesn't.

The Problem Without Leader Epoch

- **1.** Leader (broker-0) writes records at offset 100. Follower (broker-1) is at offset 98.
- **2.** Broker-0 crashes. Broker-1 becomes leader at offset 98.
- **3.** New writes arrive at broker-1: offsets 98, 99, 100... (different data than broker-0 had).
- **4.** Broker-0 recovers. Its offsets 98-100 have DIFFERENT data than broker-1's 98-100.
- **5.** Without leader epoch: broker-0 thinks it's caught up (same offsets). **Data divergence**.

The Fix With Leader Epoch

- **1.** Broker-0 was leader with epoch=5. Broker-1 becomes leader with epoch=6.
- **2.** Broker-0 recovers and sends OffsetsForLeaderEpochRequest: 'What's the end offset for epoch=5?'
- **3.** Broker-1 responds: 'Epoch 5 ended at offset 98.'
- **4.** Broker-0 truncates its log to offset 98 (removes divergent records 98-100).
- **5.** Broker-0 fetches from broker-1 starting at offset 98. Logs are now consistent.

```
# Leader epoch stored in: leader-epoch-checkpoint file
# Each partition directory contains:
# /var/kafka/data/order-events-0/leader-epoch-checkpoint
# Content:
# 0              <- version
# 3              <- number of entries
# 3 0            <- epoch=3 started at offset 0
# 4 15230        <- epoch=4 started at offset 15230
# 5 28901        <- epoch=5 started at offset 28901
```

■ **Pro Tip:** Leader epoch replaced the old high-watermark-based truncation which had edge cases causing data loss. If asked 'how does Kafka prevent data loss during leader failover?', the answer is: leader epoch-based log truncation + ISR membership + min.insync.replicas.

Q88. How does follower fetching work? What is the difference between HW, LEO, and LSO?

Hard Internals

Answer:

Acronym	Full Name	Definition	Who Uses It
LEO	Log End Offset	The offset of the NEXT record to be written. Each replica has its own LEO (leader may be ahead).	Internal: broker tracks per-replica replication progress.

HW	High Watermark	The highest offset that ALL ISR replicas have replicated. Records below HW are 'committed'.	Consumers: can only read up to HW. Records above HW are invisible.
LSO	Last Stable Offset	The lowest offset of any open transaction. Records above LSO may belong to uncommitted txn.	read_committed consumers: can only read up to LSO.

Follower Fetch Cycle

- **1.** Follower sends FetchRequest to leader with its current LEO.
- **2.** Leader responds with new records + leader's HW.
- **3.** Follower appends records to its log, advances its LEO.
- **4.** Follower updates its local HW to min(leader's HW, follower's LEO).
- **5.** Leader tracks each follower's LEO. Advances HW when ALL ISR members have replicated.

```
// Example timeline:
// Leader LEO=100, Follower-A LEO=97, Follower-B LEO=95
// ISR = {Leader, Follower-A, Follower-B}
// HW = min(100, 97, 95) = 95  -> consumers can read up to offset 94

// Follower-B fetches, catches up to LEO=100
// HW = min(100, 97, 100) = 97  -> HW advances to 97

// Follower-A fetches, catches up to LEO=100
// HW = min(100, 100, 100) = 100  -> HW advances to 100
// Now consumers can read all 100 records.

// Consumer (read_committed, with open transaction at offset 90):
// LSO = 90  -> consumer only sees up to offset 89
// Transaction commits -> LSO advances to HW -> consumer sees all
```

■ **Follow-up:** 'Why can consumers only read up to HW?' If consumers could read above HW, they'd see records that only exist on the leader. If the leader crashes before followers replicate, those records are LOST — but consumers already processed them. HW ensures consumers only see durable (replicated) records.

Q89. How do you implement a Kafka metrics interceptor for application-level monitoring?

Medium Monitoring

Answer:

```
// Comprehensive Metrics Interceptor using Micrometer
public class MicrometerProducerInterceptor
        implements ProducerInterceptor<String, String> {

    private final MeterRegistry registry = Metrics.globalRegistry;
    private Timer sendLatency;
    private Counter sendSuccess, sendFailure;

    @Override
    public void configure(Map<String, ?> configs) {
        String clientId = (String) configs.getOrDefault("client.id", "unknown");
        sendLatency = Timer.builder("kafka.producer.send.latency")
            .tag("client_id", clientId).register(registry);
        sendSuccess = Counter.builder("kafka.producer.send.success")
            .tag("client_id", clientId).register(registry);
        sendFailure = Counter.builder("kafka.producer.send.failure")
            .tag("client_id", clientId).register(registry);
    }

    @Override
    public ProducerRecord<String, String> onSend(ProducerRecord<String, String> record) {
        record.headers().add("X-Send-Nanos",
            Long.toString(System.nanoTime()).getBytes());
        return record;
    }

    @Override
    public void onAcknowledgement(RecordMetadata meta, Exception ex) {
        if (ex != null) {
            sendFailure.increment();
        } else {
            sendSuccess.increment();
            // Calculate end-to-end produce latency from header
        }
    }
    @Override public void close() {}
}

// Consumer interceptor — track processing lag
public class MicrometerConsumerInterceptor
        implements ConsumerInterceptor<String, String> {
    @Override
    public ConsumerRecords<String, String> onConsume(ConsumerRecords<String, String> records) {
        for (ConsumerRecord<String, String> r : records) {
            long producedAt = r.timestamp();
            long lag = System.currentTimeMillis() - producedAt;
            Metrics.globalRegistry.timer("kafka.consumer.e2e.latency")
                .record(Duration.ofMillis(lag));
        }
        return records;
    }
    @Override public void onCommit(Map<TopicPartition, OffsetAndMetadata> o) {
        Metrics.globalRegistry.counter("kafka.consumer.commits").increment();
    }
}
```

■ **Pro Tip:** End-to-end latency (producer timestamp → consumer processing) is the **most actionable metric** for business stakeholders. Combine interceptor metrics with Kafka JMX metrics in Grafana for a complete observability dashboard: produce rate, consumer lag, e2e latency, error rate.

Q90. How do you gracefully shut down a Kafka consumer in Spring Boot?

Medium Spring Boot

Answer:

```
// Spring Kafka handles graceful shutdown automatically via:
// 1. KafkaListenerEndpointRegistry stops all containers on context close
// 2. Each container calls consumer.wakeup() -> breaks out of poll loop
// 3. Pending records finish processing (based on container config)
// 4. Final offset commit before close

// Custom shutdown hook for additional cleanup:
@Component @RequiredArgsConstructor @Slf4j
public class KafkaShutdownHandler {
    private final KafkaListenerEndpointRegistry registry;

    @PreDestroy
    public void onShutdown() {
        log.info("Initiating Kafka graceful shutdown...");

        // Stop all listener containers
        registry.getListenerContainers().forEach(container -> {
            log.info("Stopping container: {}", container.getListenerId());
            container.stop();
        });

        log.info("All Kafka listeners stopped");
    }
}

// Kubernetes: configure shutdown grace period
spec:
  terminationGracePeriodSeconds: 60  # Give time for consumer to:
  # 1. Finish processing current batch
  # 2. Commit offsets
  # 3. Leave consumer group cleanly

// application.yml fine-tuning:
spring:
  kafka:
    listener:
      # How long to wait for in-flight records to complete on shutdown
      shutdown-timeout: 30s    # Wait up to 30s for current batch
      # For batch listener: finish entire batch before shutdown
      ack-mode: manual_immediate  # Or BATCH for full batch completion
```

■■ **Trap:** Without graceful shutdown, the consumer doesn't commit its last offsets → rebalancing gives the partitions to another consumer → records are re-processed. Always ensure *terminationGracePeriodSeconds* in K8s > your longest processing time.

Q91. What is max.poll.records? How does it interact with max.poll.interval.ms?

Medium Consumer

Answer:

Config	Default	What It Controls
max.poll.records	500	Max records returned per poll() call. Controls batch size for processing. Smaller = more frequent polls, less work per cycle.
max.poll.interval.ms	300000 (5 min)	Max time between poll() calls before consumer is considered dead. Exceeding this triggers REBALANCE and partition revocation.

The Critical Interaction

You must process all *max.poll.records* records within *max.poll.interval.ms*. If processing 500 records takes > 5 minutes, the consumer is ejected from the group.

```
// Scenario: Each record takes 1 second to process (external API call)
// max.poll.records=500, max.poll.interval.ms=300000 (5 min)
// Processing 500 records = 500 seconds = 8.3 minutes > 5 minutes!
// Result: REBALANCE! Consumer kicked out of group.

// Fix Option 1: Reduce batch size
props.put("max.poll.records", 100);  // 100 * 1s = 100s < 300s OK

// Fix Option 2: Increase poll interval (not ideal)
props.put("max.poll.interval.ms", 600000);  // 10 min

// Fix Option 3: Process asynchronously (best for slow consumers)
// Poll quickly, submit records to a thread pool, ack after completion
// Use pause()/resume() to control flow

// Fix Option 4: Use @RetryableTopic for external service calls
// Failures don't block the poll loop
```

■ **Pro Tip:** Formula: **max.poll.records * avg_processing_time_per_record < max.poll.interval.ms**. Always measure your P99 processing time and set max.poll.records accordingly. For external API calls, prefer non-blocking retry (@RetryableTopic) over blocking retry.

Q92. How do you perform a zero-downtime Kafka cluster upgrade (rolling upgrade)?

Medium Operations

Answer:

Step-by-Step Rolling Upgrade

- **1. Set inter.broker.protocol.version** to current version on ALL brokers (prevents new protocol features).
- **2. Upgrade one broker at a time:** Stop broker → install new version → start with old inter.broker.protocol.version.
- **3. Verify each broker:** Check ISR recovery, under-replicated partitions = 0, consumer lag stable.
- **4. After ALL brokers upgraded:** Remove *inter.broker.protocol.version* override. Restart brokers one-by-one.
- **5. Bump *log.message.format.version*** after clients are updated (allows new message format features).

```
# Phase 1: Upgrade binaries (keep old protocol)
# On each broker, one at a time:
# server.properties:
inter.broker.protocol.version=3.5  # Keep current version!
log.message.format.version=3.5     # Keep current version!

# Stop broker-0:
kafka-server-stop.sh
# Install new Kafka version (e.g., 3.7)
# Start with old protocol config:
kafka-server-start.sh server.properties

# Verify before moving to next broker:
kafka-metadata.sh --snapshot /var/kafka-logs/__cluster_metadata/... \
  --broker-count  # All brokers registered
# Check: UnderReplicatedPartitions == 0

# Phase 2: Enable new protocol (after ALL brokers upgraded)
# Remove inter.broker.protocol.version and log.message.format.version
# Rolling restart all brokers again
```

■■ **Trap:** Never skip versions (e.g., 3.3 → 3.7). Kafka supports upgrade from N to N+1 only. Multi-version jumps require sequential upgrades. Also: upgrade brokers FIRST, then clients. Never upgrade clients before brokers.

Q93. How do you perform Kafka capacity planning? What are the key formulas?

Hard Operations

Answer:

Key Formulas

```
# 1. STORAGE per day
storage_per_day = msg_rate * avg_msg_size * 86400 * replication_factor
# Example: 10K msg/s * 1KB * 86400s * 3 RF = 2.59 TB/day

# 2. TOTAL STORAGE
total_storage = storage_per_day * retention_days
# Example: 2.59 TB/day * 7 days = 18.1 TB

# 3. NETWORK BANDWIDTH (per broker)
inbound_per_broker = (msg_rate * avg_msg_size) / num_brokers
outbound_per_broker = inbound_per_broker * (replication_factor - 1 + num_consumers)
# Inbound: producer writes
# Outbound: follower replication + consumer reads

# 4. PARTITION COUNT
partitions = max(
    target_throughput_MB / per_partition_throughput_MB,  # Throughput-based
    num_consumer_instances                               # Parallelism-based
)
# Rule of thumb: 10-30 MB/s per partition (depends on msg size)

# 5. BROKER COUNT
brokers = max(
    total_storage / storage_per_broker,
    total_network / network_per_broker,
    total_partitions * RF / partitions_per_broker_limit  # ~4000 per broker
)
```

Workload	Msg Rate	Msg Size	RF	Retention	Est. Storage	Brokers
Small	1K/s	500B	3	7 days	~900 GB	3
Medium	10K/s	1 KB	3	7 days	~18 TB	5-7
Large	100K/s	1 KB	3	7 days	~180 TB	15-20
Extreme	1M/s	500B	3	3 days	~380 TB	30-50

■ **Pro Tip:** Always over-provision by 30-50% for burst capacity. Monitor disk usage, network throughput, and CPU. Scale out brokers BEFORE you hit limits. Tiered Storage (KIP-405) dramatically reduces storage costs for long retention.

Q94. What are the common Kafka anti-patterns and how do you avoid them?

Medium Best Practices

Answer:

Anti-Pattern	Why It's Bad	Correct Approach

Using Kafka as a database	Kafka lacks indexing, querying, and per-key lookups. Not designed for random access reads.	Use Kafka for streaming + project to DB/Elasticsearch for queries (CQRS).
Large messages (>1 MB)	Strain broker memory, slow replication, GC pressure, breaks batch efficiency.	Claim-check pattern: store large data in S3/MinIO, send reference in Kafka.
Too many partitions	Each partition uses file handles, memory, and increases rebalance time. >200K total degrades.	Start with 6-12 partitions. Scale up based on actual throughput needs.
Unbounded retries	Infinite retry on transient errors blocks the partition, violates max.poll.interval.ms.	Set max retries + exponential backoff + DLT. Use @RetryableTopic for async.
Ignoring consumer lag	Silent data loss when lag exceeds retention. No alerting means you discover too late.	Monitor lag in Grafana. Alert on growing trend. Scale consumers proactively.
Topic per customer	Thousands of topics, each with low volume. Massive metadata overhead, slow controller.	Single topic, partition by customerId. Or tenant-prefixed topics for multi-tenancy.

The Claim-Check Pattern for Large Messages

```
// Anti-pattern: send 50MB file through Kafka
// producer.send("files-topic", fileId, hugeBinaryData); // BAD!

// Correct: Claim-Check Pattern
public void publishLargeDocument(Document doc) {
    // 1. Upload binary to object storage
    String s3Key = s3Client.putObject("docs-bucket", doc.getId(), doc.getData());

    // 2. Send lightweight reference to Kafka
    ClaimCheck reference = new ClaimCheck(doc.getId(), s3Key, doc.getSize());
    kafkaTemplate.send("document-events", doc.getId(), reference);
}

// Consumer: fetch from S3 using the reference
@KafkaListener(topics = "document-events")
public void consume(ClaimCheck ref) {
    byte[] data = s3Client.getObject("docs-bucket", ref.getS3Key());
    processDocument(ref.getDocumentId(), data);
}
```

■ **Pro Tip:** These anti-patterns are **excellent interview material**. Interviewers love hearing about claim-check for large messages and CQRS for query patterns. It shows real-world production experience, not just textbook knowledge.

Q95. Design a multi-datacenter Kafka architecture for disaster recovery. What are the trade-offs?

Hard System Design

Answer:

Topology	Architecture	RPO	RTO	Complexity
Active-Passive	Primary handles all traffic. MM2 replicates to DR. Failover = redirect traffic.	Minutes (replication lag)	Minutes to hours	Low. Simple failover. Need DNS switch.
Active-Active	Both DCs handle traffic. MM2 replicates bidirectionally. Each DC has local consumers.	Near-zero (both have data)	Seconds (already serving)	High. Conflict resolution, avoid loops.
Stretch Cluster	Single cluster spans DCs. Rack awareness for replicas. Synchronous replication.	Zero (sync replication)	Seconds (automatic failover)	Highest. Needs low-latency DC interconnect.

```
# Active-Passive with MirrorMaker 2:
#
# [Primary DC: NYC]                [DR DC: Virginia]
# Kafka Cluster (3 brokers)  --->  Kafka Cluster (3 brokers)
#   order-events (12P, RF=3)  MM2    nyc.order-events (12P, RF=3)
#   payment-events            MM2    nyc.payment-events
#   user-profiles             MM2    nyc.user-profiles
#
# MM2 syncs: topics + consumer group offsets + configs
# RPO = replication lag (~1-5 seconds typical)
#
# Failover procedure:
# 1. Stop MM2 replication
# 2. Verify DR cluster has latest data (check checkpoint topics)
# 3. Rename topics: nyc.order-events -> order-events (or update consumers)
# 4. Switch DNS/load balancer to point to DR cluster
# 5. Start consumers in DR DC
# 6. Verify consumer group offsets translated correctly

# Active-Active: both DCs produce and consume locally
# MM2 config to prevent replication loops:
replication.policy.class=org.apache.kafka.connect.mirror.DefaultReplicationPolicy
# Prefixes: nyc.order-events, virginia.order-events
# Each DC ignores its own prefixed topics
```

■ **Follow-up:** 'Which topology for a fintech company?' Active-Passive for most cases (simpler, proven). Active-Active only if both regions need to independently serve traffic with local latency. Stretch cluster only if DCs are <10ms apart (same metro area) and you need zero RPO.

Q96. Design an end-to-end real-time data pipeline with Kafka for analytics.

Hard System Design

Answer:

```
# Architecture: Sources -> Kafka -> Processing -> Sinks -> Analytics
#
# [Sources]          [Kafka]           [Processing]        [Sinks]
# PostgreSQL ---+                    +---> Kafka Streams ---> Elasticsearch
#   (Debezium)  |                    |       (enrich,        (real-time search)
#               +--> Kafka Topics ---+        aggregate)
# REST APIs --->|   (Avro + SR)      |                  +---> Snowflake
#   (producer)  |                    +---> Flink/Spark        (batch analytics)
#               |                    |       (complex
# IoT Sensors -+                    |        analytics)   +---> Redis
#   (MQTT->Kafka)                    |                       (real-time cache)
#                                    +---> S3 Sink Connector
#                                           (data lake)
```

Implementation Details

- **Ingestion:** Debezium CDC for databases (zero-impact). REST APIs via Kafka producer. IoT via MQTT-Kafka bridge.
- **Schema governance:** Confluent Schema Registry with FULL_TRANSITIVE compatibility. All topics use Avro.
- **Stream processing:** Kafka Streams for simple enrichment/aggregation. Apache Flink for complex event processing (CEP), windowed joins across streams.
- **Sinks:** Elasticsearch Sink Connector for real-time search. S3 Sink for data lake (Parquet format, hourly partitioning). JDBC Sink for Snowflake/BigQuery.
- **Data quality:** Schema validation at producer, DLT for malformed records, data lineage tracking via headers.

■ **Pro Tip:** This is a **Staff/Principal-level interview question**. Key differentiators: (1) CDC over dual-writes for ingestion. (2) Schema Registry for governance. (3) Multiple processing layers (Streams for simple, Flink for complex). (4) Data lake + real-time serving = Lambda-lite architecture.

Q97. What is the difference between KafkaTemplate.send() and KafkaTemplate.sendDefault()?

Easy Spring Boot

Answer:

Method	Topic Source	When to Use
send(topic, key, value)	Topic specified explicitly in the method call	Multiple topics from one producer. Dynamic topic routing.
sendDefault(key, value)	Uses the default topic set via spring.kafka.template. default-topic property	Single-topic producers. Simplifies code when all messages go to one topic.

```
// send() – explicit topic
kafkaTemplate.send("order-events", orderId, orderJson);
kafkaTemplate.send("audit-log", orderId, auditJson);

// sendDefault() – uses configured default topic
// application.yml:
// spring.kafka.template.default-topic: order-events
kafkaTemplate.sendDefault(orderId, orderJson);  // Goes to "order-events"

// Both return CompletableFuture for async handling:
kafkaTemplate.send("order-events", orderId, orderJson)
    .whenComplete((result, ex) -> {
        if (ex != null) log.error("Send failed: {}", ex.getMessage());
        else log.info("Sent to P{} offset {}",
            result.getRecordMetadata().partition(),
            result.getRecordMetadata().offset());
    });
```

■ **Follow-up:** 'Which is preferred?' *send()* with explicit topic is clearer and more maintainable. *sendDefault()* saves a parameter but hides the destination. Use *sendDefault()* only in single-purpose services that always write to one topic.

Q98. How do you debug a stuck Kafka consumer that shows zero consumption?

Medium Debugging

Answer:

Systematic Debugging Checklist

- **1. Check consumer group state:** *kafka-consumer-groups.sh --describe --group G*. Is it 'Stable', 'Rebalancing', or 'Empty'? Empty = no active members.
- **2. Check partition assignment:** Are partitions assigned to this consumer? If concurrency > partitions, some threads are idle.
- **3. Check auto.offset.reset:** If set to 'latest' and no committed offset, consumer starts at end — appears stuck if no new messages arrive.
- **4. Check deserialization errors:** Without ErrorHandlingDeserializer, a poison pill silently crashes the consumer thread. Check logs for SerializationException.
- **5. Check max.poll.interval.ms:** If processing takes too long, consumer is kicked from group repeatedly. Look for 'consumer rebalance' in logs.
- **6. Check network/security:** Consumer might not reach broker (firewall, SASL auth failure). Check for connection timeout errors.
- **7. Check topic existence:** If *auto.create.topics.enable=false* and topic doesn't exist, consumer subscribes but never receives data.

```
# Step 1: Is the consumer group active?
kafka-consumer-groups.sh --bootstrap-server broker:9092 \
  --describe --group order-group
# Look for: STATE, CURRENT-OFFSET, LAG

# Step 2: Are there records in the topic?
kafka-run-class.sh kafka.tools.GetOffsetShell \
  --broker-list broker:9092 --topic order-events
# Shows latest offsets per partition

# Step 3: Check consumer thread state (Spring Boot Actuator)
# GET /actuator/health -> check Kafka health indicator
# GET /actuator/metrics/kafka.consumer.records.consumed.total

# Step 4: Consumer logs to grep for:
# "partition revoked"  -> continuous rebalancing
# "CommitFailedException" -> max.poll.interval exceeded
# "SerializationException" -> deserialization poison pill
# "AuthenticationException" -> SASL/ACL issues
```

■ **Pro Tip:** 90% of 'stuck consumer' issues are: (1) auto.offset.reset=latest with no new messages, (2) continuous rebalancing from max.poll.interval violation, or (3) deserialization errors without ErrorHandlingDeserializer. Check these three first.

Q99. What is the difference between Kafka, RabbitMQ, AWS SQS, and Pulsar? When to use which?

Medium Architecture

Answer:

Feature	Kafka	RabbitMQ	AWS SQS	Apache Pulsar
Model	Distributed log. Consumer pulls. Messages retained.	Message broker. Broker pushes. Messages deleted.	Managed queue. Pull-based. Messages deleted.	Distributed log. Segmented storage. Messages retained.
Throughput	Millions/sec. Horizontally scalable with partitions.	Tens of thousands/s. Limited by single node memory.	Unlimited (managed). AWS scales auto. Higher latency.	Millions/sec. Similar to Kafka. Multi-tenant native.
Ordering	Per-partition guarantee.	Per-queue (single consumer). FIFO possible.	FIFO queues offer ordering (limited throughput).	Per-partition like Kafka.
Replay	Yes — consumers can re-read any offset (killer feature)	No — message deleted after ack. No replay without DLQ.	No — message deleted after processing. No replay.	Yes — like Kafka. Cursor-based. Segmented storage.

Best for	Event streaming, log aggregation, CDC, CQRS, high throughput pipelines	Task queues, RPC, request-reply, complex routing, low-latency tasks	Simple cloud-native queuing, serverless triggers (Lambda), low-ops overhead	Multi-tenant SaaS, geo-replication, Kafka alternative with built-in storage

Decision framework: Need replay + high throughput + event streaming? **Kafka**. Need task queue + complex routing + RPC? **RabbitMQ**. Need zero ops + AWS-native + serverless? **SQS**. Need multi-tenancy + tiered storage built-in? **Pulsar**.

■ **Follow-up:** 'Can I replace Kafka with SQS?' For simple queuing, yes. But SQS lacks: replay, consumer groups, stream processing, compacted topics, exactly-once, and Kafka Connect/Streams ecosystem. If you need ANY of these, stay with Kafka.

Q100. Design a complete Kafka-based e-commerce event-driven microservice architecture.

Hard System Design

Answer:

```
# COMPLETE E-COMMERCE ARCHITECTURE
#
# [API Gateway] --> [Order Service] ---> order.created --------+
#                   (PostgreSQL)    ---> order.updated         |
#                   (Outbox + CDC)  ---> order.cancelled       |
#                                                              v
# [Payment Service] <--- order.created              [Kafka Cluster]
#   (Stripe API)    ---> payment.completed              (3 brokers, RF=3)
#                   ---> payment.failed                 (Schema Registry)
#                                                       (Avro format)
# [Inventory Svc]  <--- payment.completed               |
#   (Redis + PG)   ---> stock.reserved                   |
#                   ---> stock.reservation.failed        |
#                                                        v
# [Shipping Svc]   <--- stock.reserved          [Kafka Streams]
#   (FedEx API)    ---> order.shipped             (Enrichment +
#                                                  Aggregation)
# [Notification]   <--- order.* (all events)           |
#   (Email/SMS)    Uses headers for routing             v
#                                                 [Analytics]
# [Search Service] <--- order.* (CQRS projection)  Elasticsearch
#   (Elasticsearch) Builds denormalized search index  + Grafana
```

Key Design Decisions

- **Outbox + Debezium CDC:** Atomic DB writes + event publishing. No dual-write problem.
- **Choreography Saga:** Order → Payment → Inventory → Shipping. Compensating events for failures.
- **CQRS:** Search Service projects events into Elasticsearch for fast queries.
- **Schema Registry (FULL_TRANSITIVE):** All events use Avro. Teams can evolve schemas safely.
- **DLT + Replay:** Every consumer has a DLT. Ops dashboard for inspection and replay.
- **Monitoring:** Consumer lag (Burrow), JMX metrics (Prometheus), E2E latency (interceptors), alerts (PagerDuty).

Topic Design

Topic	Partitions	RF	Cleanup	Retention	Key
order.created	12	3	delete	7 days	orderId
payment.completed	12	3	delete	7 days	orderId
stock.reserved	12	3	delete	7 days	orderId
order.shipped	12	3	delete	30 days	orderId
user.profiles	6	3	compact	infinite	userId
product.catalog	6	3	compact	infinite	productId
*.retry-0 / *.dlt	3	3	delete	30 days	original key

Production Configs

```
# Producer: safety + performance
acks=all, enable.idempotence=true, compression.type=lz4
linger.ms=10, batch.size=32768, retries=Integer.MAX_VALUE

# Consumer: reliability
enable.auto.commit=false, isolation.level=read_committed
auto.offset.reset=earliest, max.poll.records=100

# Spring Kafka: ErrorHandlingDeserializer + @RetryableTopic
# Static group membership for K8s rolling deploys
# Batch listeners for high-volume topics (sensor data, clickstream)

# Cluster: 3 brokers, min.insync.replicas=2
# unclean.leader.election.enable=false
# auto.create.topics.enable=false
# KRaft mode (no ZooKeeper)
```

■ **Pro Tip:** This is the **ultimate Kafka interview answer**. It demonstrates: Outbox + CDC, Saga pattern, CQRS, Schema Registry, DLT handling, monitoring strategy, and production configs — all in one cohesive architecture. Practice drawing this on a whiteboard.

■ **CONGRATULATIONS!** You've completed all **100 Kafka Interview Questions & Answers** across 6 parts! This guide covers: Core Architecture, Producers, Consumers, Spring Boot, Avro & Schema Registry, Kafka Streams, Kafka Connect, Security, Monitoring, Design Patterns (Outbox, Saga, CQRS, CDC), Performance Tuning, Operations, Kubernetes, Testing, and System Design. Good luck with your interviews! — **TechTribers** (@techtribes)